Developer Foundations Study Guide

Preparation for the Claude Certified Developer – Foundations exam: the five prep-path courses condensed into concepts, decision rules and production traps, with practice questions, a timed mock exam and hands-on labs.

Exam **CCDV-F**: **53** items in **120** min, pass at **720** of 1,000

5 modules **774** minutes of course time **3** supporting courses **108** practice questions

Course content read **26 Oct 2026**

Module	Time	What it trains you to decide
M1 · MSO Foundations	57 min	Tokens and the context budget, why output varies, model tier vs. reasoning mode, how many examples to give, sync vs. streaming vs. batch.
M2 · Production-Grade Prompting, Agents & Tool Use	209 min	Which prompt technique a failure points to, when thinking pays, tool schemas and the tool-use loop, streaming safely, context and memory, agent wiring and HITL, images/PDFs/batches.
M3 · Claude Code, MCP & Integration	142 min	Permission modes and settings, CLAUDE.md vs. rules vs. hooks vs. subagents, skills and plugins, MCP transport and scope, enterprise authentication.
M4 · Production Engineering, Evals & Security	211 min	Evals and calibrated judges, test levels and tracing, retriable vs. terminal errors, model routing, cost levers, multi-agent cost, prompt-injection defense.
M5 · Accelerators & IP Contribution	155 min	Packaging for reuse, contributions a maintainer accepts, requirements and lifecycle gates, platform choice and version pinning, multi-component trust boundaries.
P1 · Building with the Claude API (optional)	≈8 h	The hands-on base the path assumes: requests, eval pipelines, prompt techniques, the tool-use loop, RAG, caching, citations, Files API and code execution, building MCP servers and clients, workflow patterns.
P2 · Introduction to Model Context Protocol (optional)	≈1 h	Building an MCP server and client in Python: tools, resources, prompts, the Inspector, and which part of the stack controls each primitive.
P3 · Model Context Protocol: Advanced Topics (optional)	≈1.1 h	Sampling, log and progress notifications, roots, the JSON message types, stdio vs. Streamable HTTP, and the stateless_http / json_response trade-offs.

Five ideas every module repeats

- **Diagnose before you add.** Each failure points to one missing structural piece: an output constraint, a tool-description exclusion, a pruning step, a deny rule. Rewording and adding length rarely fix anything.
- **Development hides production failures.** Fixtures are 3–5× smaller than real tool output, sessions are short, traffic never hits a rate limit, and the only content the agent reads is content you wrote. Measure against the largest real input before shipping.
- **Enforce with structure, not with prose.** Schemas over "please return JSON", hooks over CLAUDE.md lines, deny rules and least privilege over trusting the prompt. A prompt instruction is never a security control.
- **The eval is the gate.** Model upgrades and downgrades, prompt changes, and version promotion all move on a measured score against a pinned baseline, changing one thing at a time.
- **Ask the worst-case question.** "What is the worst outcome if this runs without a human check?" decides permission modes, HITL checkpoints, and where a seam needs a control.

Product details (model names, beta headers, limits, prices) are time-sensitive. The course itself says to confirm them against platform.claude.com and code.claude.com when you build.

The exam

From the official exam guide (v1.0, exam code CCDV-F) and the certification FAQ.

The exam guide calls its blueprint the authoritative scope of the exam. Its weights don't follow the prep path's structure, so use them to decide where your study hours go.

Items and time	**53 items in 120 minutes** (≈2.3 min each). Plan for about 135 minutes in the seat, including check-in and a short survey
Item types	Multiple-choice and scenario-based **multiple-response**. Each item says how many answers to pick
Scoring	Scaled 100–1,000; **720 passes**. It's measured against a fixed standard, not against other candidates. The report shows pass/fail, your scaled score and your % correct per domain. You see your score on screen right away

Rules	Closed book: no notes, docs, AI assistants or browser translation. A clear desk, one screen, no phone or watch. You accept an NDA before starting
Delivery	Pearson VUE, either online with OnVUE or at a test center. English or Japanese
Fee	$125. Select, Preferred and Global Premier partners get 50% off automatically; Global Premier is 100% off through 31 Dec 2026. A retake costs the full fee again
Retakes	Wait 14 days after the first fail, 30 after the second, 90 after the third. Up to 4 attempts per rolling 12 months
Validity	12 months. On-time renewal is a free, unproctored assessment; if it lapses, you retake the full exam
Practice exam	The official practice exam was retired when delivery moved to Pearson; only the guide's three sample questions remain. The timed mock below fills that gap

Blueprint: where the points are

Domain	Weight	≈ items	Study in this guide
D1 · Agents and Workflows	14.7%	8	M2 · M4 · P1
D2 · Applications and Integration	33.1%	18	M5 · M2 · P1
D3 · Claude Code	3.1%	2	M3
D4 · Eval, Testing, and Debugging	2.6%	1	M4
D5 · Model Selection and Optimization	16.8%	9	M1 · M4
D6 · Prompt and Context Engineering	11.0%	6	M2 · P1
D7 · Security and Safety	8.1%	4	M4 · M3
D8 · Tools and MCPs	10.6%	6	M2 · M3 · P2 · P3

Two things stand out. **Applications and Integration is a third of the exam**, and it includes software engineering basics (REST, JSON, async, version control, code review, refactoring) at 7.4% on their own. **Claude Code (3.1%) and evals (2.6%) are small** despite having whole modules in the path. Don't skip them, but don't over-invest either.

On the blueprint, light in the prep path

These skills are named in the blueprint, but the prep courses cover them only briefly or not at all. Here's a short primer on each. Confirm details in the official docs.

Blueprint skill	What to know	Domain
Built-in tools vs. custom tools vs. Skills vs. MCP	**Built-in** tools are Anthropic-defined (web search, code execution, text editor). **Custom** tools are functions your app defines and runs. **Skills** are reusable instructions and procedures loaded on demand. **MCP** servers make connectivity to an external system reusable across many apps. Match the job: a procedure → Skill, a shared system → MCP, logic in one app → custom tool	D8 · 4.1%
Client-side vs. server-side tools; approval patterns	Client tools are executed by your code (you get `tool_use` and return `tool_result`). Server tools such as web search and code execution run on Anthropic's side and come back as result blocks. Put an approval step before any tool that can do something irreversible	D8
Agentic frameworks (Strands, LangGraph, PydanticAI)	They're abstractions over the same tool-use loop. The message rules (`tool_use` → `tool_result`, unchanged thinking blocks, a context budget) still apply underneath. Choose one for orchestration features or for what your team already uses, not because it changes how Claude behaves	D1
Manager/supervisor hierarchies	A lead agent plans and delegates to specialist subagents, each with its own context and tools, then synthesizes their results. It's the orchestrator-worker pattern from M4, with the same cost multiplier	D1
Claude Code headless, streaming and auto modes; slash commands; sessions	Headless mode (`claude -p "…"`) runs a single prompt with no interactive UI, for scripts and CI, and can output JSON or stream it. Auto mode is the classifier-gated permission mode (M3). Custom slash commands are now Skills. Session hygiene means `/clear` between unrelated tasks and `/compact` near the limit	D3 · D2
Software engineering foundations	REST and JSON, async I/O (await and concurrency, e.g. `AsyncAnthropic` + `asyncio.gather` under rate limits), version control for prompts and config, code review of AI output (trust what the diff proves, test anything about runtime), and refactoring in small, reviewable steps	D2 · 7.4%
Breaking changes across model releases	On upgrade, expect parameters that used to work to start failing (non-default sampling params, `budget_tokens`) and the shape of outputs to change. Pin model IDs and re-run your evals before promoting	D5

Blueprint skill	What to know	Domain
Fast mode, effort levels, adaptive thinking	These are reasoning and speed settings, chosen separately from the model tier. Effort controls how much adaptive thinking happens. Confirm per-model support in the docs	D5
WebSockets and SDKs that wrap REST	The SDKs are thin layers over REST (auth, retries, typing). Streaming uses server-sent events over HTTP. In a web app, you push updates to the browser yourself (SSE, WebSockets or polling)	D5 · 6.1%
Output handling: defensive parsing	Validate structure and values, handle `refusal` and `max_tokens`, and be skeptical of confident output: check numbers and cited facts against the source	D6
PII and data leakage	Minimize or redact PII before it reaches prompts and logs, restrict who can read logs, and pick a platform and retention setup (ZDR or BAA where required) that matches the data	D7

Answering scenario questions

- **Read the constraint word first.** "Most effective", "must never", "lowest cost", "no user waiting" or "reusable across apps" usually settles the answer before you read the options.
- **Watch for these distractors.** All of them appear in the guide's samples and the course checkpoints:
  - "add a line to the prompt" when the question needs enforcement
  - "use a bigger or smaller model" for a problem that isn't about the model
  - "change the temperature" for anything that isn't sampling
  - "raise `max_tokens`" for a context problem
  - "retry" for a terminal error
  - "paste the data into context" when live, reusable access is needed
- **Pick the structural fix over the wording fix**: schema over instruction, hook over CLAUDE.md, least privilege over trust, Batches over a loop.
- **For multiple-response items**, select exactly the number asked for, and judge each option on its own merits.
- **Pace yourself.** At about 2.3 minutes per item, mark long scenarios and come back to them rather than sinking five minutes in on the first pass.

Exam-day checklist

When you book

- Sign in with your partner company email. A domain that isn't on the partner record takes 7–10 days to add.
- Make sure your Pearson profile name matches your ID *exactly*. Request corrections at least 24 h ahead (they take 24–48 business hours).
- Request accommodations at least 10 days ahead, before scheduling.
- You must be 18+ with a government-issued photo ID.

The week before (online)

- Run the OnVUE system test on the *same* computer and network you'll use.
- Ask IT to allow Pearson's domains (`*.onvue.com`, `*.pearsonvue.com`, `*.pvue1.com` and the others on the setup page).
- A managed corporate laptop often blocks the exam. If so, use a personal computer on a personal network, or book a test center.
- Rescheduling: the exam guide says 24 h, the FAQ says 48 h. Plan on **48 h** to be safe.

On the day

- Close the apps that block the exam application. On Windows that includes Edge, Chrome, Teams, Zoom, Outlook, Discord, remote-desktop tools, Windows Terminal and the Claude desktop app.
- Clear the desk: no notes, phone, watch, headphones or second monitor.
- Stay in camera view; don't speak to anyone.
- Not showing up or arriving late forfeits the fee.

After

- Your score appears on screen; a pass brings a Credly badge email within minutes.
- If you fail, the per-domain percentages show what to review before the 14-day retake window.
- Add a personal email to Credly so you keep the badge if you change jobs.
- Appeals go to Pearson within 14 days.

Readiness by domain

Your accuracy per exam domain, combining the section quizzes, the drill and your last mock, weighted by how much each domain counts. The "Study next" list points to where an hour of review is worth the most points.

Cheat sheet

Context window

- Input already too large: validation error *before* generation.
- Fits on input, hits the ceiling mid-output: partial output with `stop_reason: model_context_window_exceeded`.
- Nothing is ever silently dropped; trimming or summarizing is the app's job.
- API models: at least 200k tokens; newest flagships (Fable included) 1M by default.

Newest-model gotchas

- Non-default `temperature`, `top_p`, `top_k` return a 400. Steer through prompting.
- Thinking is adaptive, tuned with an effort setting. `budget_tokens` is deprecated and returns a 400.
- Thinking content is omitted by default; request summarized display to show it.

`stop_reason` values

- `end_turn`: finished, no action requested.
- `tool_use`: run the tool calls and send results.
- `max_tokens`: truncated, may be mid-structure.
- `refusal`: HTTP 200, but fail fast and log.
- `model_context_window_exceeded`: ran out of window.

Error classes

- Retriable: `429` rate limit, `529` overloaded, `500 502 503 504`, most timeouts.
- Terminal: `400 401 403 404`.
- Honor `retry-after`; otherwise exponential backoff with jitter and a cap.
- When unsure, treat it as terminal. The SDK already retries transient errors, so don't stack loops.

Message Batches API

- Up to 100,000 requests or 256 MB per batch.
- Submit, get `batch_id`, poll, download. Up to 24 h.
- About 50% cheaper per token (confirm at build time).
- Results come back unordered: match on `custom_id`.

Prompt caching

- `cache_control: {type: "ephemeral"}` on the last block to cache; up to 4 breakpoints, or automatic mode.
- Order processed: tools → system → messages.
- TTL 5 min, refreshed on each hit; `ttl: "1h"` opt-in.
- Writes 1.25× (5 min) or 2× (1 h); reads 0.1×.
- Exact prefix match; minimum about 1,024 tokens on most models.

Structured outputs

- JSON outputs: `output_config.format` with type `json_schema`.
- Strict tool use: `strict: true` on the tool definition.
- First call on a new schema is slower; the grammar is cached 24 h.
- Adds input tokens, can't be combined with prefill, and you still check for `refusal` and `max_tokens`.

Images and files

- Image tokens ≈ [w/28] × [h/28]. 1000×1000 ≈ 1,296.
- One-off: inline base64. Reused: Files API `file_id` (beta, not on Bedrock/Vertex). Public stable URL: URL source.
- PDFs use a `document` block; `title` and `context` are optional.

Beta headers named in the course

- MCP connector: `mcp-client-2025-11-20`
- Managed Agents: `managed-agents-2026-04-01`
- Skills on the API: `code-execution-2025-08-25` + `skills-2025-10-02`

Claude Code settings

- User `~/.claude/settings.json` · Project `.claude/settings.json` · Local `.claude/settings.local.json` (git-ignored) · Enterprise `managed-settings.json`.
- Deny beats allow in every mode. An enterprise deny can't be overridden.
- Hooks: `PreToolUse` exit code 2 blocks; precedence deny > ask > allow.

MCP placement

- stdio = local subprocess. Streamable HTTP = remote/shared. SSE = legacy.
- Local scope `~/.claude.json` · User · Project `.mcp.json` · Enterprise managed.
- The API connector only reaches remote HTTP servers.
- Secrets: `${ENV_VAR}` references, never inline.

Compliance routing

- Managed Agents sessions live server-side: not ZDR or HIPAA BAA eligible.
- Direct API has no EU data residency; use Bedrock or Vertex with the region pinned.
- FedRAMP: Claude for Government (High), Bedrock GovCloud (High, IL4/5), Vertex Assured Workloads. AWS Marketplace Enterprise is not.
- ZDR eligibility varies by model and platform.

Numbers worth remembering

- Orchestrator-worker ≈ 15× the tokens of a chat.
- Production tool output ≈ 3–5× test fixtures.
- Tool descriptions: 3–4 sentences, including when *not* to use.
- CLAUDE.md starts to dilute past about 200–300 lines.
- Claude Code loads MCP tools upfront only under about 10% of the window (opt-in).

Model choice

- Tiers: Fable (most capable) · Opus · Sonnet (default) · Haiku (speed/cost).
- Start on Sonnet. Go up only when an eval shows a miss; go down only when an eval shows the drop is acceptable.
- Pin full IDs. From Claude 4.6, the ID alone pins a snapshot; older models need the date suffix.

Study toolkit

The module sections present the content in course order. This part reorganizes it for review: a study method, tables that line up the same idea across courses, diagrams of the mechanisms that are hard to follow in prose, and a drill that mixes questions from every section.

How to study with this guide

Step 1

Test first

Answer a section's practice questions *before* reading it. Your misses tell you which parts to read closely. Being tested is what makes material stick, not rereading it.

Step 2

Redraw the mechanisms

After reading a section, close it and sketch the matching diagram below from memory. Then compare. Gaps in your sketch are gaps in your understanding.

Step 3

Explain each trap out loud

For each "production trap" box, say the symptom, the real cause and the fix in one sentence. The symptom → cause → fix table below is the answer key.

Step 4

Mix and space

Each day, run a round of the mixed drill. It interleaves sections, which is closer to the exam, and brings back questions you missed more often than ones you know.

Step 5

Cover and recall

In the final days, cover the right-hand columns of the tables and the cheat sheet, then recall them. Re-read only what you couldn't produce.

Day	Read	Then
1	This toolkit, the cheat sheet, M1	Pretest M1 and M2; draw the tool-use loop
2	M2: prompting, thinking, tool use	Draw the streaming timeline; drill round
3	M2: streaming, context, agents, memory, multimodal	Draw the context budget and the escalation ladder; drill round
4	M3	Recite the enforcement ladder; drill round
5	M4	Draw the defense layers; drill round
6	M5	Draw the lifecycle gates; drill round

Day	Read	Then
7	P1 · Building with the Claude API	Check the out-of-date table twice; drill round
8	P2 and P3 · MCP	Draw the Streamable HTTP connections; drill round
9	Symptom → cause → fix table, coverage matrix	Cover and recall; two drill rounds
10	Cheat sheet	Redraw all seven diagrams; drill until most cards are "Known"

Mixed drill

Symptom → real cause → fix

Every module teaches through failures. This is the whole set in one place. Many look like one problem and are really another, which is exactly what the exam probes.

You see	The real cause	The fix	From
Output shape varies: casing, labels, stray sentences	No output constraint	Exact format plus few-shot examples, or structured outputs	M2
The prompt gets longer each pass, and the output gets longer and slower	Adding description instead of diagnosing; the model mirrors a verbose input	Name the failure type and add the one missing technique	M2
Claude picks the wrong one of two similar tools	Overlapping descriptions	When-to-use and when-not-to-use sentences; merge them with a type parameter	M2
Tool choice degrades after a fixed number of turns	Tool output has crowded the instructions out of the window	Prune results, compact early, measure real tool-output sizes	M2
A validation error on the *retry* request	A dropped stream committed a half-built `tool_use` to history	Append only after `message_stop`; discard the partial turn	M2

You see	The real cause	The fix	From
"tool_result references unknown tool_use_id"	The result's id doesn't match the call's id	Echo the `tool_use` id exactly	M2
The request is rejected after you trimmed history	A thinking block was removed or edited, so its signature broke	Return thinking blocks unchanged	M2
A write that passed validation breaks a downstream system	No pause between "change proposed" and "write committed"	HITL checkpoint before any irreversible tool	M2
The agent stalls around session four	In-context memory kept growing across sessions	External storage, injecting only a summary	M2
A "batched" nightly job hits rate limits	Chunking a list and looping over the sync API	Message Batches API	M2
Files outside the task's scope get deleted	`bypassPermissions` removed every prompt	Deny rules first; `auto` instead of bypass	M3
A rule that's in CLAUDE.md gets ignored	Dilution in an 800+ line file	Trim it, move path rules to rules files, make critical rules hooks	M3
A plugin works only for its author	Absolute paths, undocumented env vars	`$CLAUDE_PROJECT_DIR`, `${CLAUDE_PLUGIN_ROOT}`, test on a clean machine	M3
A key leaks even though "we removed it"	An inline secret in a committed `.mcp.json` stays in history	Rotate it, reference `${VAR}`, add a PreToolUse guard	M3
OAuth works in staging, fails in production	Redirect URIs are registered per host	Register the production URI; check for per-environment apps	M3
Every check passed, but the value is wrong	Validation checks shape; there was no graded set	An eval with edge cases, run on every change	M4
Unit and functional tests pass, end-to-end fails	A format mismatch in the handoff between components	Agree on the handoff format and add an integration test	M4

You see	The real cause	The fix	From
The first traffic peak takes the feature down	No error path, then instant retries	Classify errors; back off with a cap; honor `retry-after`	M4
The bill tripled and quality didn't move	Fanning out a tightly coupled task	Single agent; fan out only independent parts	M4
The agent wrote a file nobody asked for	Fetched content was followed as instructions	Treat it as data, plus a PreToolUse hook on writes	M4
The parser broke with no deploy	A model alias advanced	Pin the full ID, keep the prior version, gate on the eval	M5
Rejected at security review over residency	Platform chosen by familiarity	Settle compliance constraints during scoping	M5
Progress and sampling vanish once deployed	`stateless_http` / `json_response` turned on	Test with the production transport; choose the flags deliberately	P3

How strongly each control holds

Ordered from a request to a guarantee. Exam scenarios that say "must never" want a control near the bottom.

Control	Lives in	What it guarantees	Use it for
Prompt instruction	System or user prompt	Nothing; it's a request that can slip on untested inputs	Behavior, tone, scope
CLAUDE.md / rules files	Claude Code context	Followed most of the time; weakens as the file grows	Project conventions
Structured outputs / strict tools	API decoding	Output matches the schema, except on `refusal` or `max_tokens`	Machine-readable output, tool arguments
Allow / deny rules	Settings (enterprise level can't be overridden)	Deterministic; deny beats allow in every mode	Paths, commands, MCP tools

Control	Lives in	What it guarantees	Use it for
PreToolUse hook	Claude Code lifecycle	Runs on every matching call; can block (exit code 2) and log	Guardrails, audit trail
Least-privilege identity	Credentials and roles	Limits what a steered agent can reach	Write paths, secrets, data access
OS-level sandbox	Process isolation	Holds even where no rule or hook covers a path or endpoint	Filesystem, network
Human-in-the-loop gate	Agent loop design	A person approves before an irreversible action runs	Writes, deletes, sends, deploys

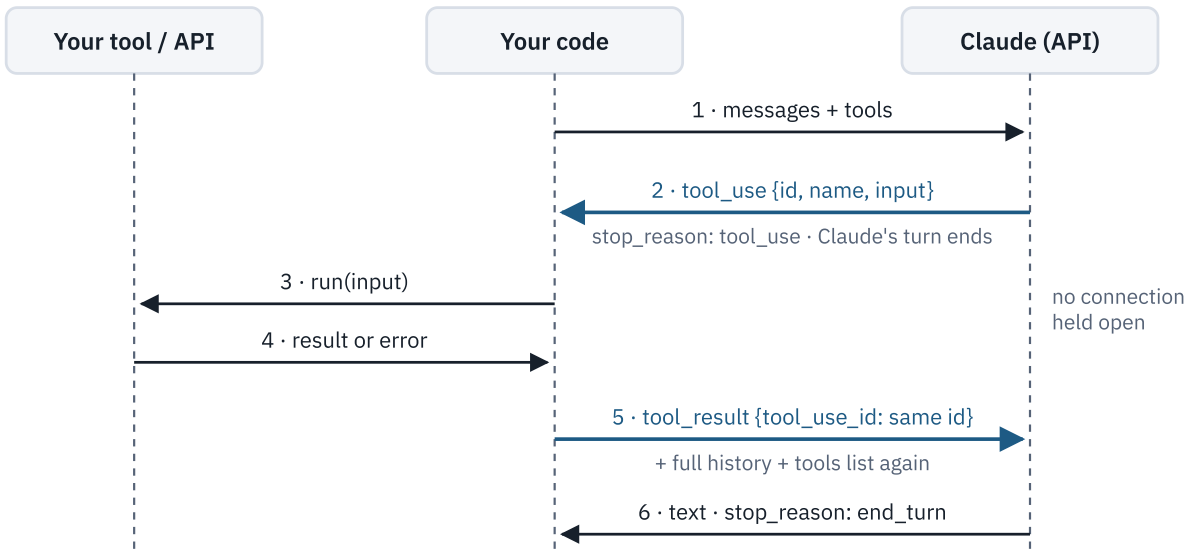
Where each topic is covered

● taught in depth · ○ mentioned or applied. Use it to revise one topic across every course in a single sitting.

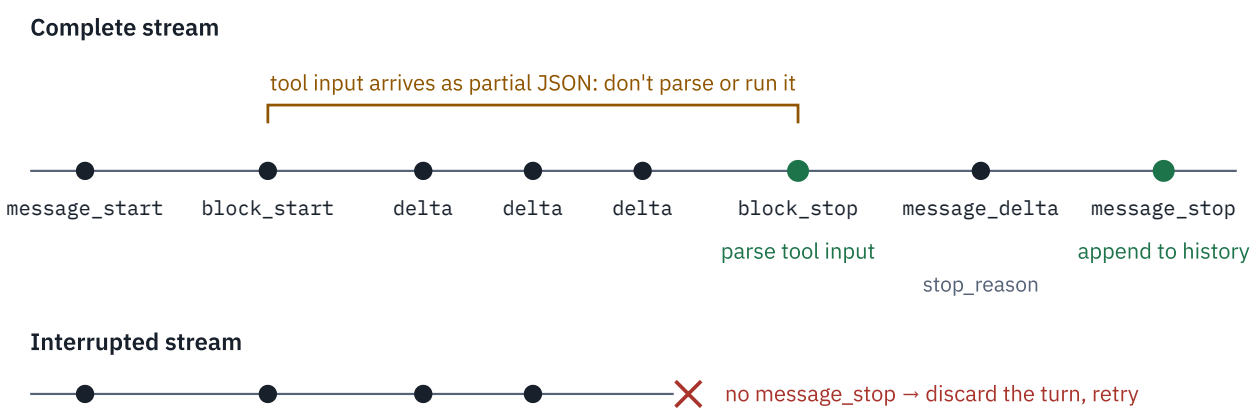
Topic	M1	M2	M3	M4	M5	P1	P2	P3
Prompt techniques	○	●				●		
Structured outputs and prefill		●				○		
Extended thinking	○	●				○		
Tool schemas and the tool loop		●		○		●		
Streaming	○	●		○		○		
Context window and compaction	●	●	○					
Prompt caching		○	○	●		●		
RAG and retrieval		○	○	○		●		
Memory, Skills, CLAUDE.md		●	●			○		
Permissions, hooks, plugins			●	○				
MCP		○	●		○	●	●	●
Evals and graders	○			●	○	●		

Topic	M1	M2	M3	M4	M5	P1	P2	P3
Errors and retries				●		○		
Model choice and pinning	●	○		●	●			
Images, PDFs, batch	○	●		○		●		
Workflows vs. agents		●		○		●		
Security and compliance		○	●	●	●			○
Platforms and deployment		○			●			

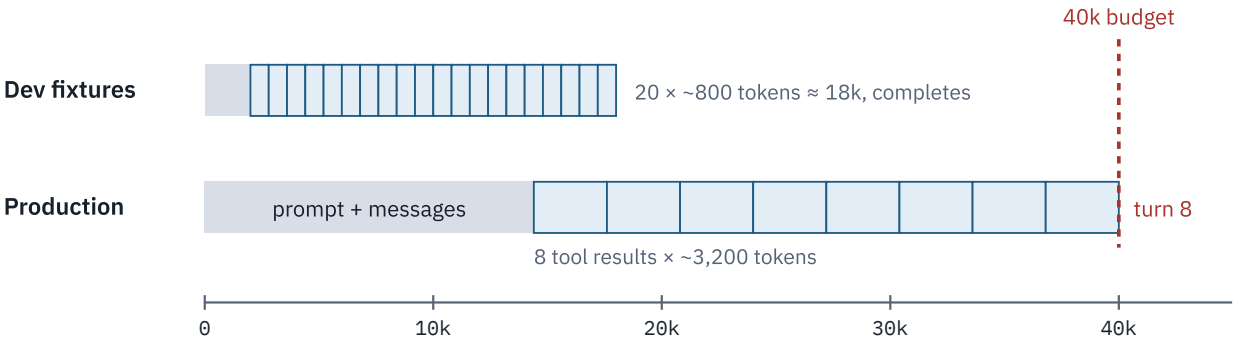
Mechanisms, drawn



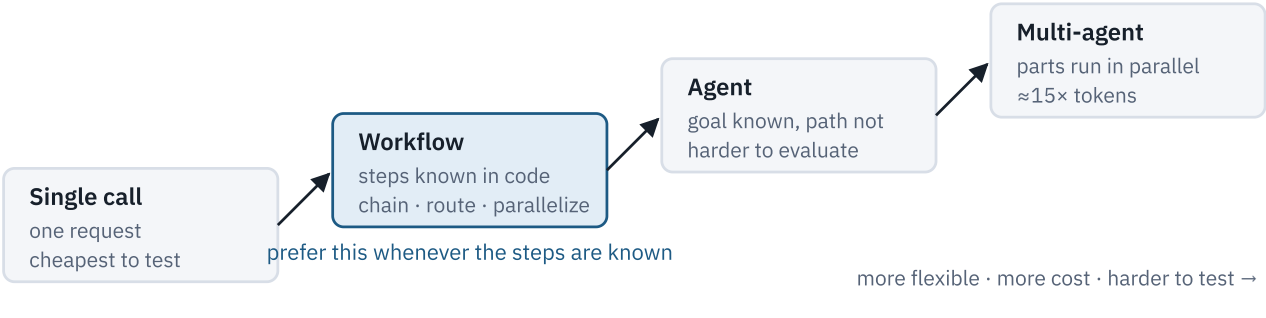
The tool-use loop. Claude only *asks* for a tool. Your code runs it and closes the loop with a fresh request whose `tool_result` carries the same id, in the very next user turn. Steps 2–5 repeat while `stop_reason` is `tool_use`. M2 · P1



Streaming: when it's safe to act. A stream ending is not a message completing. "block" is short for `content_block_*`. Parse tool input only at its `content_block_stop`, and write the turn to history only at `message_stop`. M2 · M4 · P1

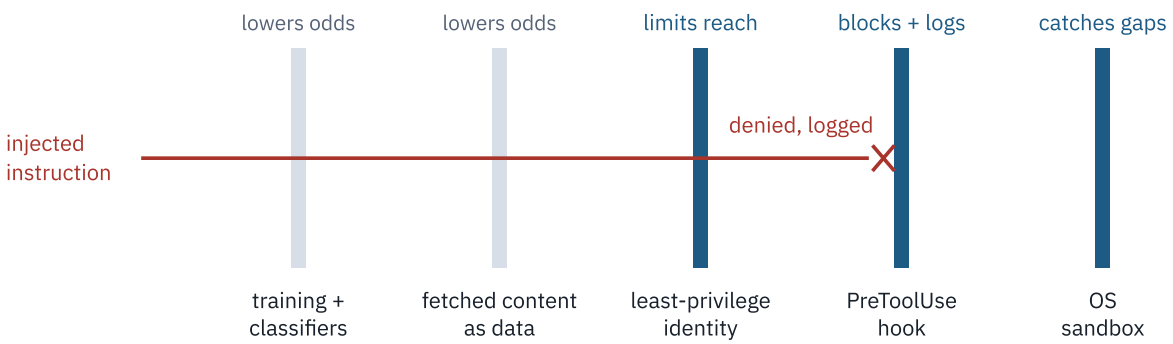


Same budget, bigger tool outputs. Production tool output was 4× the fixtures, so the receipts agent hit its 40k budget at turn 8, crowding out the instructions. It looked like bad tool selection. Fix it by pruning results and compacting early, and by measuring real outputs before you ship. M2

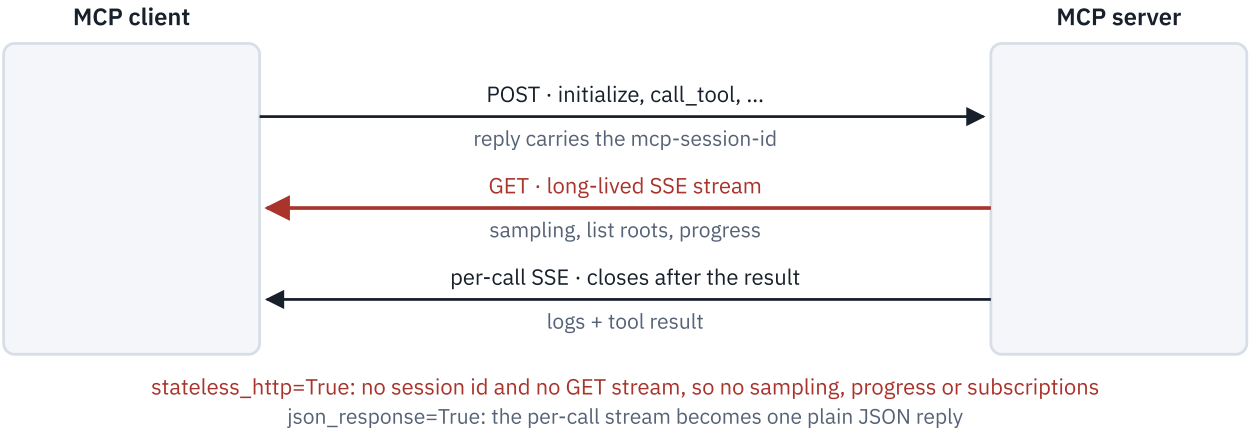


Climb only when you must. Each step up handles more variation, at the price of cost, predictability and testability. Workflow patterns (chaining, routing, parallelization, evaluator-optimizer) cover most production needs. Fan out to several agents only when the work splits into independent parts.

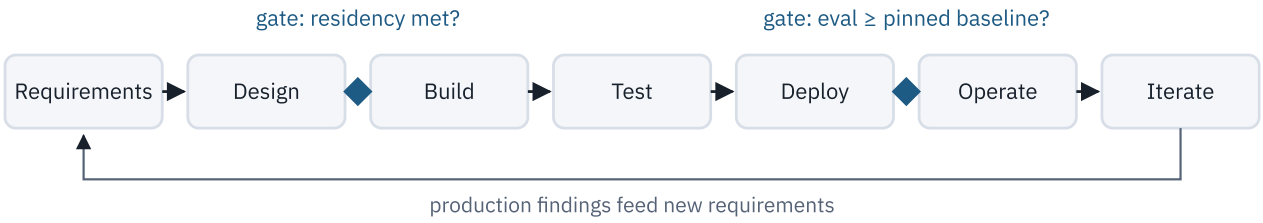
M2 · M4 · P1



The reliable boundary is on the action side. Training, classifiers and delimiters reduce how often an injection lands, but none of them is guaranteed. What a steered agent can actually do is bounded by its identity, by a hook that denies (deny > ask > allow) and logs, and by the OS sandbox for anything no rule covers. M4 · M5



How a server talks back over HTTP. On stdio either side writes whenever it likes. Over HTTP the server can only push through SSE streams the client has opened, and the two flags switch those streams off. Load balancers are the usual reason teams turn on `stateless_http`. P3 · M2 · M3



Gates are where a regulated project stays in control. Don't start building until the platform meets the residency requirement, and don't promote a version until it beats the pinned eval baseline on a slice of traffic. M5 · M4

MSO Foundations

Module 1 · 57 min

The vocabulary the rest of the path assumes: what you are paying for, why the same prompt answers differently, which levers are independent, and how requests reach the model.

Tokens and the context budget

- Everything is counted in tokens: system prompt, history, tool definitions, tool results and the response. Tokens are both the billing unit and the window's unit.
- Characters-per-token depends on the model's tokenizer and changes between generations, so treat any rule of thumb as model-specific.
- The window is a fixed budget holding the whole request at once. It rarely fills in development and fills fast in production.

Sampling and non-determinism

- At each step the model samples the next token from a probability distribution. Low temperature concentrates it and high temperature spreads it out.
- Temperature 0 makes runs more repeatable but never guarantees identical output. On the newest models, sampling parameters can't be changed at all.
- Consequence for testing: never assert exact text. Assert properties (a field exists, a value is in range, it parses) and use a model-graded eval for meaning.

Two independent levers

Lever	What it sets	Rule
Model choice	Which family member runs: Fable, Opus, Sonnet, Haiku	Start on Sonnet; move up or down only on eval evidence.
Reasoning mode	Whether and how deeply the model thinks, per request (adaptive thinking + effort)	Pays off on hard multi-step work; wasted on lookups and classification.
Prompting mode	Zero-shot, one-shot, or multi-shot (few-shot) examples	Add examples when the output's shape, casing or edge case keeps missing. One or two examples often beat another paragraph of instructions.

These levers interact: a stronger model may succeed zero-shot where a smaller one needs examples, and examples can let a cheaper model do the job. Try the smallest model with the fewest examples that pass your eval.

How requests reach Claude

Pattern	Use it when
Raw REST vs. SDK	Same API and model. The SDK adds auth, request building, retries and parsing.
Synchronous	Short responses, backend jobs, nobody watching.
Streaming (SSE)	Long responses or a user watching. You reassemble the pieces.
Async client	Concurrency without blocking. Python has `AsyncAnthropic`; the TypeScript client is already Promise-based, so there's no separate class.
Message Batches	Bulk offline work where cost matters more than turnaround: pipelines, eval runs.

Production-Grade Prompting, Agents & Tool Use

Module 2 · 209 min

This is the largest module. Every topic is framed as a failure that testing misses and production exposes, paired with the implementation decision that prevents it.

Prompting craft: diagnose, then add one technique

What you see	What's missing
Wrong shape: a sentence instead of a label, prose instead of JSON	**Output constraint**: format, field names, types, length, preamble or not, what to do when data is absent
Scope or tone drifts, and gets worse deeper into the conversation	**System prompt**, or a more specific one: role, format and rules that hold across turns
Right task, but invented structure	**Few-shot examples** wrapped in consistent tags such as `<sample_input>` / `<ideal_output>`
Clean on tested inputs, breaks on a variant	A **constraint or example covering that variant**

- **XML tags** mark the boundary when instructions and data mix (for example `<my_code>` and `<docs>`). Descriptive names work; there are no official tag names.
- You can take few-shot examples from your highest-scoring eval outputs.
- Don't stack all four techniques on a simple job: "summarize this paragraph" needs neither examples nor a schema.

Production trap · the prompt that grew longer

Six revisions each added description and none added the missing constraint. Along the way the verbose prompt produced verbose output (over 2,000 characters per call) and latency went up with no accuracy gain, because the model matches its length and style to the input. The fix was a JSON schema and two examples. If three re-prompts in a row fail, stop adding text and diagnose.

Structured outputs: moving the format into the API

A prompt asks for a format; constrained decoding enforces it. **JSON outputs** (`output_config.format`, type `json_schema`) constrain the final response. **Strict tool use**

(`strict: true`) validates the arguments Claude passes to a tool. The costs: a slower first call while the grammar compiles (cached 24 h), a few injected input tokens, no prefill, and responses that still won't match on `refusal` or `max_tokens`.

Extended thinking

Task	Decision
Multi-step reasoning with several constraints: derivations, multi-hop logic, dependent plans	Enable it, with effort matched to depth
Classification, format conversion, extraction, short facts	Leave it off; a constrained prompt is cheaper and just as accurate
Agent loops that plan across tool calls	Enable it and budget for the planning step

Carry-back rule

In tool-use conversations, every thinking block (redacted ones included) goes back to the API unchanged on the next turn. Each block is signed; editing, summarizing or dropping it gets the request rejected. Thinking tokens are billed as output tokens.

Tool use and schema design

Claude never runs your tools. The loop: you define tools → Claude replies with a `tool_use` block (name, id, input) and `stop_reason: tool_use` → **your code** executes → you send a `tool_result` with the same id in a new request → Claude continues or ends the turn. The model is stateless between calls.

Block	Rule your code must follow
`text`	When it comes alongside `tool_use`, keep the full content array in history.
`tool_use`	Answer it in the *immediately following* user turn. Resolve every tool_use from one turn together.
`tool_result`	Sent in the user role. `tool_use_id` must match exactly (matching is by id, not position). Set `is_error: true` on failure.
`thinking`	Return it unchanged.

- **Name**: specific (`get_account_balance` , not `get_data`).
- **Description**: 3–4 sentences covering what it does, when to use it, *when not to*, and what it returns. Claude routes mostly on name plus description; parameter types are a secondary signal.
- **Required vs. optional**: mark a field required only when the call makes no sense without it. Requiring everything forces Claude to invent values.
- **Dependencies**: dependent calls go in separate turns. Independent calls can run in parallel in one turn, which current models do by default; `disable_parallel_tool_use` forces one call per turn.
- If two tools need ever-longer descriptions to stay apart, merge them into one tool with a type parameter.
- Exclusion conditions such as "only if X was already called" rely on the full history being sent. Truncate history and they silently stop working.

MCP instead of writing schemas yourself

- The loop doesn't change. The client sends `ListToolsRequest` and hands the definitions to Claude.
- Connected servers' definitions use context even when idle. With the API connector, use `mcp_toolset` with `default_config` and per-tool configs: `defer_loading` and `enabled` .
- Transports: stdio for local servers, Streamable HTTP for remote (SSE-only is deprecated). The API connector supports only remote HTTP servers.
- Use both: an MCP server for breadth, an allowlist to narrow it, and tuned descriptions for the tools you actually route to.

Streaming without corrupting state

Event	Handler action
`message_start`	Create an empty content array
`content_block_start`	Open a slot at the index (tool_use arrives with name and id, no input yet)
`content_block_delta`	Append the text, partial JSON or thinking fragment
`content_block_stop`	Finalize. Tool input becomes parseable only now
`message_delta`	Record `stop_reason` and final usage

Event	Handler action
`message_stop`	The message is complete. Only now append it to history

Production trap · the dropped stream

A handler appended the turn whenever its read loop ended. A network blip cut a `tool_use` block mid-JSON, and the retry failed validation, so the team spent an afternoon debugging the schema. A loop ending is not a message completing. Gate the append on `message_stop`, discard partial turns, and retry from the last complete turn.

```
if stop_seen:
    messages.append({"role": "assistant", "content": assemble(blocks)})
else:
    raise StreamInterruptedError("partial turn discarded; retry from last complete tur
```

Context engineering

Strategy	Use when	You lose
Pruning (rewind)	After an unproductive path or a debugging detour	Anything learned after the rewind point
Compaction (`/compact`; server-side compaction in the API, beta)	Near the ceiling while still on the same task	Whatever the summary leaves out
Clearing (`/clear`; a new API session)	The next task is unrelated	Everything not persisted (for example in CLAUDE.md)
Subagent handoff	A self-contained task, especially noisy exploration	Visibility into the intermediate steps

- **Prompt caching** is the highest-leverage cost cut for multi-turn sessions with a stable system prompt and tools. **Token counting** (`count_tokens`) measures a request without running inference, so you can gate requests before they error.
- For manual compaction, the summarizer prompt has to say what to keep: modified file paths, decisions made, errors and how they were fixed. A vague summarizer is one of the most common causes of multi-session failures.
- **RAG can break in three places**: chunking (too small or too large; use overlap), embedding match (semantic search can miss exact identifiers, so add lexical search), and assembly (if

chunks arrive in the wrong structure, the model answers from memory). A fetch-once index is inspectable but is infrastructure to maintain; iterative agentic search skips the index but costs more tokens per query.

Production trap · fine in dev, ceiling in prod

Receipts averaged ~800 tokens per tool result in testing and ~3,200 in production, so a self-imposed 40k budget filled at turn 8. The symptom looked like bad tool selection: unpruned outputs had crowded out the system prompt. When tool choice degrades after a fixed number of turns, check the window before the schema.

Building the agent

Choose a workflow when…	Choose an agent when…
You can write the exact steps in code	You know the goal and tools but not the path
Errors are costly and you want step-level guardrails	Non-determinism is acceptable and the toolset bounds what the agent can do
Standard observability is required	Inputs vary unpredictably
Every run follows the same sequence	The task needs creative sequencing of tools

Escalate in order: a single call, then a workflow, then an agent. Agents need transcript-level observability.

Wiring path	Who runs the loop	Check before committing
Raw Messages API loop	You: loop, tools, context, retries, exits	All maintenance is yours
Agent SDK (Python/TS)	The SDK, in your process; you still execute tools	Set `settingSources` explicitly: `["user","project","local"]` to match the CLI, or `[]` for isolation
Claude Managed Agents (beta)	Anthropic runs the loop and sandbox; the agent is a versioned API resource used by ID	Server-side sessions are not ZDR or HIPAA BAA eligible. Moving from SDK prototype to Managed Agents means re-expressing the agent, not exporting it

- The four steps on any path: register tools, scope the system prompt to the task and its tools, run the loop (return a result for every call), and define exit conditions that don't

depend on Claude volunteering to stop.

- **HITL placement**: before destructive calls (write, delete, send) for high risk, after a planning step for medium, and on unexpected output (error flag, empty or out-of-range result).
- **Over-tooling** is the more common failure: start with the minimum toolset.

Constraint	Route that usually survives review
Attorney-client privilege	Direct API/SDK inside the firm's app, SSO, an approved LLM gateway with full request/response logging. You log at the app layer, since content isn't captured for you
HIPAA	A BAA-covered configuration (a dedicated HIPAA-enabled org) or Bedrock/Vertex on a HIPAA-eligible account. The BAA excludes Console, Workbench, beta features and consumer plans
GDPR / residency	Bedrock or Vertex with the region pinned in client config. The direct API currently has no EU residency
FedRAMP	Claude for Government, Bedrock GovCloud, or Vertex Assured Workloads. Keep dev/test off the commercial endpoint too
Internal policy	Build on the cloud vendor the CIO already approved

Production trap · the agent that edited production

A config agent changed a rate limit to a value its validator accepted, then exited cleanly. Downstream services broke, because the exit condition only checked the file. The missing piece was a pause between "change proposed" and "write committed". Any tool that can do something irreversible needs a checkpoint, designed in from the start.

```
if block.name == "update_record":
    show_proposal(block.input)
    if input("Approve? (yes/no): ").strip().lower() != "yes":
        tool_results.append({"type": "tool_result", "tool_use_id": block.id,
                             "content": "Update rejected by operator."})
        continue
result = execute_tool(block.name, block.input)
```

Memory scope and Skills

Scope	Fits	Cost / loss
In-context	Short sessions that don't need to survive a restart	Token cost grows every turn; lost when the session ends
External storage	Continuity across sessions, users or instances	Retrieval latency plus read/write code
Summarized	Long conversational agents	Detail the summarizer prompt didn't keep
Stateless	Independent jobs that finish and close	All prior context

- Decide memory scope at design time. The module estimates about 20 minutes then versus about an hour of refactoring under production pressure.
- **Skills** (`SKILL.md`: frontmatter name and description, then instructions) load on demand. Only name and description sit in context until a request matches. CLAUDE.md loads every session; in-context instructions grow and vanish when the session ends.
- Subagents don't inherit Skills or history, but they *do* inherit the parent's permission context. List the skills a subagent needs explicitly.

Images, PDFs and batch

- Run the image-token formula on the largest production image, not on test thumbnails. A resize step is often the whole fix.
- For visual prompts, say how to handle ambiguity: "If objects overlap, describe each separately and note the overlap."
- Chunking a list and looping over the synchronous API is still serial calls, with the same rate limits. Batching is a different way of submitting work, not a smaller loop.
- Never put batch in a user-facing flow.

Claude Code, MCP & Integration

Module 3 · 142 min

Making a working integration configurable, shareable and safe once it leaves your machine. Every convenience that works locally (a lenient mode, a growing CLAUDE.md, an absolute path, an inline key, staging-only OAuth) breaks somewhere else.

Permission modes

Claude Code works in three phases: explore, then plan, then code. Plan mode holds it in exploration until you approve.

Mode	Auto-approves	Right for
`default`	Reads only	New or unfamiliar codebases
`acceptEdits`	Reads, edits, and `mkdir touch rm rmdir mv cp sed` inside the working directory; protected paths still prompt	Trusted local work where shell commands still need review
`plan`	Reads; proposes without editing	Sensitive or unfamiliar code
`auto`	Everything, but a classifier screens each action. Blocks prod deploys and migrations, mass deletes, credential exfiltration and force-push to main by default	Fewer prompts with a safety net (research preview)
`dontAsk`	Only allow-listed tools plus read-only; denies everything else with no prompt	Locked-down CI and scripts
`bypassPermissions`	Everything; also drops the protected-path guard. Only `rm -rf /` and `rm -rf ~` still prompt	Disposable, isolated containers or VMs only

Placing the human gate

Let cheap, reversible actions through. Gate anything hard to undo or touching a sensitive path, enforced with a deny rule, or with default or plan mode. For code your team marks sensitive, the agent is never the only reviewer. The mode sets the session default; the gate overrides it for the one action that costs too much to get wrong.

Production trap · bypass mode

A "simple rename" in `bypassPermissions` ran a cleanup script whose pattern also matched `/deploy/config/prod/` and deleted three production files without a prompt. The prompt that would have caught it was on the script invocation; `acceptEdits` would have let a plain `rm` inside the working directory through. Set deny rules on sensitive directories *before* loosening the mode, and prefer `auto` to bypass.

```
{ "permissions": {
    "allow": ["Bash(npm run:*)"],
    "deny":  ["Bash(rm:*)", "Bash(git push:*)", "Read(.env.production)"] } }
```

Durable project context

Mechanism	Loads	Put here
CLAUDE.md	Every session, before your first message (`/init` drafts one)	Universal constraints, test commands, framework decisions, no-touch paths
`.claude/rules/*.md`	Only when files matching the frontmatter `paths` glob are touched. With no `paths` , every session. Subfolders are just organization	Path-specific guidance (for example, SQL transaction rules for `src/db/**/*.sql`)
Hooks	Run a script at a lifecycle event and add nothing to context	Guardrails that must always hold, formatters, audit logging
Subagents	A fresh, isolated context; returns only a summary	Exploration and parallel work that would bloat the main context

- **Hook events**: `PreToolUse` (can block with exit code 2, reason on stderr), `PostToolUse` (can't block; formatting, tests, audit), `UserPromptSubmit` , `Stop` , `Notification` (permission needed or idle 60 s), `SessionStart` , `SessionEnd` .
- The built-in **Explore** and **Plan** subagents skip CLAUDE.md and git status for speed; **general-purpose** loads both. Custom subagents in `.claude/agents` need skills listed in their frontmatter.

Production trap · the 847-line CLAUDE.md

A no-touch rule at line 347 was ignored, buried under style guides, decision logs and archived notes. CLAUDE.md is a working set, not an append log. Move path rules into rules files and history into reference docs, audit once it passes a few hundred lines, and turn the rule you can't afford to dilute into a hook.

Skills, commands and plugins

Runtime	How a skill loads	Watch for
Claude Code	Discovered in `.claude/skills`; loads on a description match or `/skill-name`	Runs under the active permission mode
Messages API	Sent with the request; runs in Anthropic's code-execution container	Needs both beta headers; no local files or tools
Agent SDK	Only if `settingSources` / `setting_sources` includes filesystem sources	Most common surprise: the skill silently never loads
Managed Agents	Listed on the agent resource; loaded server-side	Attached when the agent is defined, not per session

- Portability rules: the description is what triggers matching; don't assume local tools exist; subagents don't inherit skills.
- Skills are now the recommended format for commands, and `.claude/commands/` is legacy. Add `disable-model-invocation: true` for explicit-only workflows.
- Plugin commands are namespaced: plugin `payments` plus command `run-tests` becomes `/payments:run-tests`. Renaming the plugin renames all of them.
- A plugin bundles skills, hooks, subagents and MCP servers through a marketplace. Add one with `/plugin marketplace add owner/repo`. Enterprises use a managed marketplace allowlist plus `extraKnownMarketplaces` to push one to everyone.
- A plugin only carries what it bundles. A deny rule the author relied on locally doesn't travel with it.

Production trap · works on the author's machine

A skill called `/Users/alexmorgan/…/validate.sh`, and another relied on an undocumented `DEPLOY_TOKEN`. Every install succeeded and every run failed. Use `$CLAUDE_PROJECT_DIR` for project scripts and `${CLAUDE_PLUGIN_ROOT}` for bundled ones, document and validate env vars at install, and test on a clean machine.

MCP servers

- A server exposes **tools** (actions), **resources** (read-only data the client pulls into context, either direct or templated by a parameter) and **prompts** (vetted instruction templates invoked by name).
- Claude Code defers MCP tool definitions by default and loads them through tool search. An opt-in mode loads them upfront when they fit in about 10% of the window.
- Per-tool permissions use `mcp__server__tool`. A deny on one tool beats an allow on the server. The connector's `enabled` flag controls whether the model sees a tool; a permission rule controls whether it may run.
- GitHub's server is remote HTTP and authenticates with a PAT sent as a Bearer header from an env var. Linear uses OAuth, which issues and stores the token for you.

Situation	Transport + scope	Secrets
Personal local tool	stdio + Local (`~/.claude.json`)	Env vars only
Shared team server	HTTP + Project (`.mcp.json`, committed)	OAuth or env vars, never committed keys
Personal experiment	stdio or HTTP + Local	Env vars only
Org-wide	HTTP + Enterprise managed settings	Admin-managed, config locked

A stdio server in `.mcp.json` looks shared, but each clone spawns its own local process and every teammate needs the runtime installed.

Production trap · the inline key

A key pasted into `.mcp.json` "for now" was committed, and within 48 hours it was on four machines and a CI runner. Removing it in a later commit doesn't clean history: treat it as compromised and rotate. Here, rotating broke two other services that shared the key. Use `"Bearer ${WAREHOUSE_MCP_TOKEN}"`, a CLAUDE.md convention, and a `PreToolUse` hook that blocks credential-looking writes to `.mcp.json`.

Enterprise integration

Service	Auth	Secret lives	Logging / lock
Remote, user identity (SaaS)	OAuth: a 401 triggers the browser flow	Token stored by the client	PostToolUse audit hook; managed settings
Remote, service identity	API key from an env var (CI injects it as a secret)	Environment or a secret store	Same
Local	stdio, file-system permissions	None needed	Deny rules in managed settings

- **Credential handling**: (1) keep values out of config files and reference them; (2) env vars for local or short-lived secrets, a secret store for shared or audited ones; (3) rotate on a schedule and immediately after exposure. Scope each credential narrowly and keep a record of who uses it.
- **Code modernization scoping**: settle blast radius, how changes are audited, and who approves each phase before the session starts. Plan mode builds confidence, hooks lock sensitive paths, and CLAUDE.md holds the target conventions so the agent doesn't drift back to legacy patterns.
- **Reviewing AI-generated code** (from the module recap): treat the review as findings to triage, not a verdict. Trust what's provable from the diff, such as a missing null check, after checking the cited lines. Treat claims about runtime or other systems as hypotheses to test. Put the human gate where a finding becomes a hard-to-reverse action, and give the reviewer your conventions.

Production trap · staging OAuth

OAuth redirect URIs are registered per host, so `staging.mycompany.com` working says nothing about production. Add the production URI, check whether the customer requires separate app registrations per environment, and put that step on the deployment checklist.

Production Engineering, Evals & Security

Module 4 · 211 min

Proving that a working agent keeps working under real traffic. Each production failure here traces back to a decision nobody wrote down.

Write the design document first

Decision	Becomes
Success criteria, specific enough to grade ("two sentences listing every action item and owner")	Eval cases
Failure handling: each error marked retriable or terminal, plus what the user sees	Error paths
Cost and latency budget: per request, monthly ceiling, latency target, reliability floor	Instrumentation targets
Trust boundary: untrusted inputs and the minimum set of actions	Least privilege enforced by hooks

Evals and judges

- An eval is cases, expected behavior and grades. Write it *before* the feature so you define success instead of rationalizing whatever came out. A first score of 2–3/10 is normal; the trend is what matters.
- Change one thing at a time (prompt, tools or model) and read the per-case results. A flat average can hide three fixes and three new breaks.
- Coverage beats perfection: 20 cases with edge inputs catch more than 3 polished ones. Let Claude generate more cases from a labeled seed set, then spot-check them.
- Read the failure category: formatting points to output instructions, wrong facts on retrieved content point to retrieval, and failures only on long input point to context handling.

Output	Grader	Blind spot
One correct label or value	Exact / string match (≈ free)	Fails valid paraphrases or reorderings

Output	Grader	Blind spot
Structured or code output	Code check: `json.loads`, `ast.parse`, ranges, required fields	Says nothing about content quality
Open-ended quality	LLM-as-judge with a rubric	Noisy and costly; meaningless until calibrated

Making a judge defensible

Have it return strengths, weaknesses and reasoning *before* the score; otherwise scores drift toward a safe ~6. Calibrate by running it on human-labeled cases and measuring agreement. If agreement is low, tighten the score definitions, add good and bad examples, and re-measure. Run code checks on every commit and the judge on a scheduled pass.

Production trap · two dates in one sentence

"Ordered March 3, received April 12" came back with April 12 as the order date. Every validation check passed, because validation checks shape, not correctness. There was no graded set to catch it. The eval detects and guards; a prompt change fixes. Ask the model to list edge cases (no date, a relative date, two dates) and turn them into graded cases.

Test levels and tracing

Level	Isolates	Misses
Unit	One function (parser, tool wrapper)	How pieces fit together
Functional	One Claude call returns the right shape	The system around it
Integration	The handoff between two components, where most silent failures live	Whole-flow behavior
End-to-end	The full user flow	Where exactly it broke

A **trace** records each step's prompt, tool calls, intermediate output and timing, which turns "the case failed" into "step 4, the parser raised `KeyError: amount`." For mixed traffic, a cheap classifier can send lookups to fetch-once retrieval and multi-part questions to agentic search.

Production trap · green parts, red seam

`retrieve()` returned a list of `{"content": …}` dicts while `build_prompt()` expected a string, so the model answered from memory. The fix is to agree on the handoff format (`"\n".join(c["content"] for c in chunks)`) and add an integration test that runs both components on real retrieved data.

Failure handling

One question: would retrying the identical request plausibly work? Rate limits and overloads clear with time; malformed or unauthorized requests never will.

Error	Handling	After that
429 rate limit	Retry: `retry-after`, else exponential backoff + jitter, capped	Clean error or a cached/simpler result
529 overloaded	Retry with backoff (server load, not your rate limit)	Fail over or degrade gracefully
400 bad request	Fail fast	Fix or reject the input
Tool error	Retry only if the cause is transient	Return `is_error: true` to Claude, never an empty result
Refusal (200, `stop_reason: refusal`)	Fail fast; your status-code check won't catch it	Raise and log

```
def call_with_retry(make_call, max_attempts=5, cap=30):
    for attempt in range(max_attempts):
        try:
            return make_call()
        except anthropic.APIStatusError as e:
            if e.status_code not in {429, 529, 500, 502, 503, 504}:
                raise                                  # terminal: fail fast
            wait = e.response.headers.get("retry-after")
            time.sleep(float(wait) if wait else min(cap, 2 ** attempt) + random.random
    raise RetryBudgetExhausted()
```

Production trap · the call that never failed in dev

A loop with no error path collapsed at the first traffic peak. Then a tight immediate-retry loop deepened the rate limit. Classify the error first, then back off with a cap.

Model selection and cost

- Defaulting to the most capable model is the most common and most expensive mistake. A higher tier can still come out cheaper if it reaches the answer in fewer tokens, and the cost of a wrong answer belongs in the calculation.
- **Routing**: send most traffic to a default model and override it on a cheap signal such as task type, input length or a difficulty label. Pin a single model when traffic is uniform.
- **Instrument every call** for input/output tokens, latency and error rate. One step often accounts for about 90% of spend.
- **Levers**: model choice, prompt and context size, number of tool calls, streaming (perceived latency), batching (cost), caching (repeated prefixes). Batching and caching stack.
- **Caching only pays** when the prefix is identical (even adding "please" breaks it), recurs within the TTL, clears the minimum length, and can tolerate being briefly stale.

Orchestrator-worker as a hiring decision

A lead plans the work, subagents explore in parallel in their own contexts, and the lead synthesizes. Anthropic's research setup (Opus 4 lead, Sonnet 4 workers) beat a single Opus 4 agent but used about 15× the tokens of a chat, so a 10k-token question becomes about 150k. Token usage explains most of the performance gain. It suits broad, independent research and not tightly coupled work like coding. Use a strong lead with cheaper workers, and give every worker its own retry handling.

Reliability floor first. Fix the latency ceiling and retry budget (for example 4 s and 3 retries), then optimize cost above them. A pinned baseline eval score is how you enforce the floor.

Security

- **Prompt injection**: the model reads its context as one stream with no built-in line between your instructions and fetched text. Any content someone else can write is a vector (pages, shared docs, DB records, email, tool output), including content planted for later or hidden in white text or images.

- Delimiters are a soft boundary, and training plus classifiers lower the odds without guaranteeing anything. The boundary you can rely on is **what the agent is allowed to do**.
- **Jailbreak** means bypassing the model's safety constraints; **injection** means hijacking your app's instructions. Both get the same defense: constrain the input and limit the actions.
- **Least privilege** decides how bad a steered agent can be. Narrow write paths and explicit denies (`/etc` , `/secrets` , `~/.aws`) turn an injection into a denied action and a log line. Anything that can edit the agent's auth config is as privileged as the secret itself.
- **Hooks** enforce; prompts don't. A `PreToolUse` hook returns `permissionDecision: "deny"` and writes an audit entry. Precedence is deny > ask > allow.
- **OS-level sandboxing**, with the filesystem limited to the working directory and the network to named endpoints, holds even where no hook or rule covers a path. Enterprise reviewers ask about it first.
- **Regulated scoping**: be ready to explain data residency, audit logging and managed configuration, and confirm ZDR eligibility for each model and platform.

Production trap · "our users are internal"

A summarizer followed a hidden line on the page it fetched and wrote a file nobody asked for. Trusting the user didn't help, because the instruction came from the content. Treat fetched content as data and put a hook in front of the write tool.

Accelerators & IP Contribution

Module 5 · 155 min

This module starts where your code starts working: making it reusable, acceptable to a maintainer, safely versioned, defensible on a platform, and bounded when its components connect.

Packaging for reuse

Asset	Parameterize	Document
Agent template (prompt, tool schemas, loop)	Prompts, paths, scopes, thresholds, credentials by reference	Environment assumptions, expected inputs, handled failures, the eval that defines "working"
MCP server package	Scopes, credentials by reference, per-customer paths	Inputs per tool, scope boundaries, failure modes
Eval suite (dataset + rubric)	Thresholds, dataset paths	Rubric logic, score meanings, the pinned baseline

Every asset also ships its audit story: what data it touches, which identity it runs as, and the log it leaves. Package while the build is fresh. Loose scripts are the classic fake template.

Production trap · fast but not reusable

The repo path, model name, thresholds and domain prompt fragments were all hardcoded to meet a deadline. The next team found nothing to configure and no eval, and rewrote it. Warning signs: no parameters, no assumptions doc, no bundled eval.

Contributing back

- **Match the channel**: the Claude Cookbook takes focused reference implementations; tools and MCP servers go to their own repositories. A whole application sent to the Cookbook stalls, so extract the pattern first.
- **What lets a maintainer verify it**: code that does one thing, a runnable example, a test, and a short statement of assumptions.
- **Rights and attribution come before technical review.** Engagement code can carry licensing limits. If you can't clear them, escalate to the owner and don't contribute.

Requirements and lifecycle

- **Functional** requirements are checkable behaviors ("draft a reply citing policy; never auto-send without approval"). **Infrastructure** requirements come from asking about latency (measured where the user is), scale, residency and identity. Those four usually decide the platform.
- **Lifecycle**: requirements → design (platform, model, boundaries) → build → test → deploy (pin, gate on eval) → operate (instrument, guardrails) → iterate. Gates sit between phases: no build until residency is satisfied, no promotion until the eval clears the baseline.

Where it runs and how it's pinned

Platform	Identity / data	Choose when	Pin with
First-party Claude API	Anthropic identity and terms; new features first	No binding cloud or residency constraint	Full model ID; keep the prior one
Claude Platform on AWS	Through the customer's AWS account; inference run by Anthropic outside the AWS boundary	On AWS but wants Anthropic IDs, lifecycle and parity	Claude API ID format
Claude in Amazon Bedrock	Messages API at `/anthropic/v1/messages`; data stays in the customer's AWS boundary	On AWS with a compliance posture there	`anthropic.`-prefixed full ID; partner retirement dates differ
Claude on Bedrock (legacy)	InvokeModel / Converse	Existing integration not yet migrated	ARN-versioned IDs
Google Vertex AI	Google Cloud IAM; regional or global endpoints	On GCP with a compliance posture there	Vertex model ID format
Third-party (e.g. Microsoft Foundry)	Hosted on Azure (some models) or hosted on Anthropic (the rest)	The customer already runs that product	Platform's own controls; confirm residency per model

- Aliases such as `opus` move and can resolve differently per platform. A pinned full ID is a fixed snapshot. From Claude 4.6 the ID alone pins; earlier models need the date suffix (`claude-haiku-4-5-20251001`).

- Version the prompt and asset alongside the code, keep the previous version for rollback, and promote through the eval on a slice of traffic.
- Compare platforms on **latency** measured from the customer's region with their payload, **compliance** (usually pass/fail; already-certified clouds win), and **total cost** (egress, platform fees and integration, not just token price). If compliance is pass/fail, it decides on its own.

Production trap · the alias moved

`model="opus"` advanced overnight, the output shape changed, the parser threw `KeyError "summary"`, and there was no pinned version to roll back to. A second trap: choosing the platform the team knew best and failing the customer's residency review after the build was finished.

Multi-component trust boundaries

Component	Boundary at its seam	Control
First-party API (entry point)	Requests entering from outside	Input validation; the identity the call runs under
Claude Code task	What it fetched is untrusted downstream	Treat it as data at the next seam
MCP server	The system access it holds	Least privilege (e.g. read-only) and access logging

An app is only as contained as its most privileged seam. Components that each pass their own tests say nothing about the seams between them. If a seam can't be secured, take it to a human owner instead of shipping around it.

```
def build_agent(repo_path):                          # parameterized
    return Agent(model="us.anthropic.claude-opus-4-8",  # pinned Bedrock ID
                 system_prompt=SYSTEM_PROMPT, repo_path=repo_path,
                 tools=[read_file, run_linter])
deploy(platform="amazon_bedrock", identity=aws_role_arn,
       retain_previous_pinned_version=True)          # rollback target
next_call(input=treat_as_data(code_task.run(fetch_url=customer_page)))
assert eval_suite.run(model="us.anthropic.claude-opus-4-8") >= baseline_score
```

Building with the Claude API

Supporting course · about 8 hours · 85 lessons

The hands-on course the Foundations path builds on. You work in Python notebooks, from a first request through evals, prompt techniques, tools, RAG, platform features and MCP, and finish with workflow and agent patterns. It is older than the path, so read the next table before you memorize any API details.

Where this course is out of date

Where it conflicts with the Foundations path, go with the path (and the current docs).

This course teaches	Current guidance
Tune `temperature` from 0 to 1 by task type	The newest models reject non-default `temperature`, `top_p` and `top_k` with a 400; steer through the prompt
Prefill ```` ```json ```` and stop on ```` ``` ```` to get raw JSON	Use structured outputs (`output_config.format`, or `strict: true` on tools). JSON outputs can't be combined with prefill
Thinking via `{"type": "enabled"}` with a token budget (min 1,024, below `max_tokens`)	Adaptive thinking tuned by effort; `budget_tokens` is deprecated and returns a 400 on the newest models
A cache "lives for one hour"	The default TTL is 5 minutes, refreshed on each hit; 1 hour is opt-in. Writes cost 1.25× or 2×, reads 0.1×
Image tokens ≈ (width × height) / 750	≈ [w/28] × [h/28], with per-model resolution limits
Model and tool version strings (`claude-sonnet-4-0`, `text_editor_20250124`, `web_search_20250305`, `code_execution_20250522`)	Look up current IDs and tool versions in the docs
MCP over "HTTP, WebSockets and other protocols"	stdio for local and Streamable HTTP for remote; SSE-only is legacy

Accessing Claude with the API

- **Always go through your own server.** The API key must never ship in client-side code. The flow runs client → your server → Anthropic API → model → back the same way.
- Every request needs an API key, `model`, `messages` and `max_tokens`. `max_tokens` is a safety cap, not a target length. Official SDKs exist for Python, TypeScript/JavaScript, Go and Ruby.
- Inside the model: tokenize → embed (numbers that capture meaning) → contextualize (refine each token by its neighbors) → generate one token at a time, mixing probability with controlled randomness.
- Generation stops at `max_tokens`, a natural end-of-sequence token, or a stop sequence. The response carries the content, `usage` (input and output tokens) and `stop_reason`.
- **The API is stateless.** Keep the message list yourself and send the full history every turn; otherwise "write another sentence" has nothing to refer to.
- System prompts go in the `system=` parameter. Include it only when it's set, because the API rejects `system=None`.
- **Streaming**: pass `stream=True` for raw events, or use `client.messages.stream(...)` with `stream.text_stream` for text only. `stream.get_final_message()` returns the assembled message for storage.

Prompt evaluation

- After drafting a prompt you can test it once, tweak a corner case or two, or score it with an eval pipeline and iterate. Only the third holds up against real users.
- Five steps: draft a prompt → build a dataset (by hand, or generated with a fast model like Haiku) → run each case through Claude → grade 1–10 → change the prompt and repeat.
- Code shape: `run_prompt(case)` → `run_test_case` returns output, case and score → `run_eval(dataset)` averages the scores.

Grader	Good for
Code	Length, required or banned words, syntax (`json.loads`, `ast.parse`, `re.compile` scored 10 or 0), readability
Model	Quality, instruction following, completeness, helpfulness, safety. Ask for strengths, weaknesses and reasoning, or scores drift to ~6
Human	Depth, comprehensiveness, conciseness, relevance. The most flexible, but slow

Define the criteria first: format and valid syntax suit code graders, task-following suits a model grader. Combine them, for example `(model_score + syntax_score) / 2`, and

weight to taste.

Prompt engineering techniques

Each technique was measured on an athlete meal-plan prompt. First scores of about 2/10 are normal.

Technique	What it means	Effect in the course
Clear and direct	Open with an instruction and an action verb ("Generate a one-day meal plan..."), not a question	2.32 → 3.92
Be specific	**Output guidelines** (length, structure, required elements) in almost every prompt; **process steps** for complex, multi-angle problems	3.92 → 7.86
XML tags	Delimit large or mixed content with descriptive names: `<sales_records>` beats `<data>`	Grows with prompt size
Examples	One- or multi-shot in `<sample_input>` / `<ideal_output>`. Cover corner cases like sarcasm, reuse top-scoring eval outputs, and say *why* an example is ideal	Fixes corner cases

Tool use

- **Tool functions**: descriptive names, validated inputs, and clear error messages. Claude reads errors and can retry with corrected arguments.
- **Schemas**: `name`, a 3–4 sentence `description` (what it does, when to use it, what it returns), and an `input_schema` with a description for every argument. You can have Claude draft the schema from your function and the docs. `ToolParam` adds type safety.
- **Responses** come back as several blocks (text plus `tool_use` with id, name and input). Append the whole `response.content` to history.
- **Results** go back as a user message of `tool_result` blocks (`tool_use_id`, string `content`, `is_error`), one per tool call. Send the `tools` list again on the follow-up request.
- **The loop**: call Claude → append → stop if `stop_reason != "tool_use"` → run every requested tool (catch exceptions as `is_error: true`) → append the results → repeat. A small router maps tool names to functions. Adding a tool means writing the function and schema, then registering both in the list and the router.

- **Streaming tool arguments**: `input_json` events carry `partial_json` and a running `snapshot`. By default the API buffers until each top-level key is complete and valid, so output arrives in bursts. **Fine-grained tool calling** turns that validation off: chunks arrive faster, but your code must survive invalid JSON.

Built-in tool	Who runs it	Notes
Text editor	**You** implement the file operations; Claude already knows the schema from a small stub	View files or directories, view line ranges, replace, create, insert at a line, undo
Web search	Anthropic (server tool)	`max_uses`, `allowed_domains`; must be enabled in Console. Returns the query, results and citations
Code execution	Anthropic, in an isolated Docker container with **no network**	Data goes in through the Files API (`container_upload` with a `file_id`), and generated files come back as downloadable file IDs

Retrieval-augmented generation

Stuffing a whole document into the prompt hits length limits, works less well on very long prompts, and costs more and runs slower. RAG chunks the document ahead of time and sends only the relevant chunks. The cost is a preprocessing step, a search mechanism, and the risk that a chunk lacks context.

Chunking	Trade-off
Size-based (+ overlap)	Cuts words and context, but works on anything, including code. The usual production default
Structure-based (split on headers)	The cleanest chunks, but only when you control the formatting
Sentence-based (+ overlap)	A practical middle ground
Semantic	The most relevant chunks, but the most expensive and complex

- **Embeddings** are lists of numbers (each −1 to 1) whose individual dimensions can't be interpreted. Anthropic doesn't offer an embedding model; the course uses VoyageAI.
- **Flow**: chunk → embed → normalize → store in a vector DB *together with the original text* → embed the query → nearest neighbors → prompt with the chunks in XML tags.

- **Cosine similarity** ranges from −1 to 1, and closer to 1 means more similar. **Cosine distance** = 1 − similarity, so lower means more similar.
- **BM25 lexical search** weights rare terms highly, which catches exact IDs like `INC-2023-Q4-011` that semantic search can miss.
- **Hybrid retrieval** merges the two rankings with reciprocal rank fusion: `RRF(d) = Σ 1 / (k + rank_i(d))`, where k is often 60.

Platform features

- **Extended thinking**: turn it on only after prompt work has stalled on your evals. Signatures prevent tampering; redacted blocks come back encrypted (flagged by safety systems) and must be passed through as-is.
- **Images**: up to 100 per request, 5 MB each, max 8000 px for a single image or 2000 px when sending several, as base64 or a URL. Use the same prompting craft as with text, such as a step-by-step counting method with a second check, or a one-shot example image.
- **PDFs**: a `document` block with `application/pdf`. Claude reads text, images, charts and tables.
- **Citations**: add `"title"` and `"citations": {"enabled": true}` to a document block. Responses then include `cited_text`, the document index and title, and page numbers (character positions for plain-text sources).
- **Prompt caching**: put `cache_control: {"type": "ephemeral"}` on a longhand block. Everything up to and including that block is cached and must match exactly. Breakpoints work on system prompts, the last tool, images and tool blocks. Processing order is tools → system → messages, with up to 4 breakpoints and a 1,024-token minimum. Check `usage` for `cache_creation_input_tokens` (a write) vs. `cache_read_input_tokens` (a read).

Building with MCP

- MCP isn't "just tool use". It changes *who* writes and maintains the tools: an MCP server wraps a service (such as GitHub) so you don't have to. Real projects usually build either a client or a server, not both.
- Messages: `ListToolsRequest` / `Result` and `CallToolRequest` / `Result`. Your server fetches the tool list through the client, sends it to Claude, and runs Claude's tool calls through the client.
- **Python SDK**: `FastMCP("DocumentMCP")`, plus `@mcp.tool(name, description)` with Pydantic `Field(description=…)`, which generates the schema for you.

- **Inspector**: `mcp dev mcp_server.py` opens a browser UI for testing tools, resources and prompts without a full app.
- **Client**: wrap the SDK's `ClientSession` so it cleans up after itself, and expose `list_tools()`, `call_tool()`, `read_resource()`, `list_prompts()` and `get_prompt()`.

Primitive	Analogy	Example
Tool	An action Claude decides to call	`read_doc_contents`, `edit_document`
Resource	A GET handler: data the app pulls straight into the prompt, with no tool call. Direct `docs://documents` or templated `docs://documents/{doc_id}`, with a `mime_type`	An `@report.pdf` mention injects the document's contents
Prompt	A tested template the server returns as messages	A `/format` command that rewrites a document as Markdown

Claude Code, workflows and agents

- **Claude Code**: `/init` writes CLAUDE.md, which can be project-level (shared), local (personal, not in git) or user-level (all projects). `/clear` resets context, and `claude mcp add <name> <command>` connects a server.
- The most effective loop: feed it the relevant files → ask for a plan (no code yet) → implement. For TDD: context → brainstorm test cases → write the tests → write code until they pass.
- **Agents need abstract, combinable tools.** Claude Code has `bash`, `read`, `write`, `edit`, `glob` and `grep`, not a "refactor" tool. A good agent also asks for missing inputs, like the purchase date before calculating a warranty expiry.
- **Environment inspection**: ask "How will Claude know this action worked?" That means screenshots after UI actions, reading a file before editing it, and checking generated output (for example, captions or frame grabs of a rendered video).

Workflow pattern	Shape	Use when
Chaining	Sequential focused steps, optionally with code in between	A long prompt keeps breaking constraints; add a second revision pass that only fixes them

Workflow pattern	Shape	Use when
Routing	One categorizing call, then *one* specialized pipeline	Inputs differ by type (educational vs. entertainment scripts)
Parallelization	Independent specialist calls run at once, then aggregated	A decision spans several independent criteria (suitability per material)
Evaluator-optimizer	A producer and a grader in a loop until the output is accepted	Output can be checked against the input (rendered CAD vs. photo)

Workflows vs. agents

Workflows are more accurate, easier to test and more predictable, but less flexible and more work to design. Agents handle novel tasks and can ask users for input, but complete tasks less reliably and are harder to evaluate. The course's advice: build workflows wherever you can, and use agents only when the task truly calls for one.

Introduction to Model Context Protocol

Supporting course · about 1 hour · 13 lessons

This course is the MCP chapter of Building with the Claude API published on its own: the same CLI document chatbot, built with both an MCP server and a client. If you've studied that chapter, the only new material is the closing review of the three server primitives below. Its final assessment loads interactively, so it isn't reproduced here.

Build checklist

1. **Server**: `mcp = FastMCP("DocumentMCP")`, with your data in memory.
2. **Tools**: `@mcp.tool(name=…, description=…)` on a typed function, with arguments described by Pydantic `Field(description=…)`. The SDK writes the JSON schema. Raise `ValueError` for bad input so Claude can see the error.
3. **Test**: `mcp dev mcp_server.py` opens the Inspector in your browser (a local URL such as `127.0.0.1:6274`). Connect, list tools, run them, and check resources and prompts.
4. **Client**: wrap the SDK's `ClientSession` in your own class so it cleans up, and implement `list_tools()` and `call_tool()`.
5. **Resources**: use `@mcp.resource("docs://documents", mime_type="application/json")` for a direct resource, or `"docs://documents/{doc_id}"` for a templated one whose parameter becomes a function argument. The client calls `read_resource(AnyUrl(uri))`, takes `contents[0]`, and parses it by MIME type.
6. **Prompts**: `@mcp.prompt(name, description)` returns a list of messages (`base.UserMessage`). The client calls `list_prompts()` and `get_prompt(name, args)`, and the CLI shows them as `/` commands.

Who controls each primitive

Primitive	Controlled by	Use it to	In Claude's own app
Tools	**The model.** Claude decides when to call them	Give Claude new capabilities	Claude running code or doing calculations
Resources	**The app.** Your code decides when to fetch and inject	Get data into the UI (autocomplete) or into context	"Add from Google Drive"

Primitive	Controlled by	Use it to	In Claude's own app
Prompts	**The user.** Triggered by a click, menu choice or slash command	Offer predefined, optimized workflows	Workflow buttons under the chat input

In short: tools serve the model, resources serve your app, and prompts serve your users.

Model Context Protocol: Advanced Topics

Supporting course · about 1.1 hours · 15 lessons

This course covers the parts of MCP where the *server* reaches back to the client (sampling, notifications, roots), the JSON messages underneath, and why moving from stdio to Streamable HTTP can silently switch those features off. The walkthrough videos and the assessment have no written content, so they aren't summarized here.

Sampling: the server borrows the client's model

- A server that needs text generation, such as summarizing the Wikipedia pages it fetched, asks the **client** to call Claude for it instead of holding its own API key.
- Benefits: a simpler server, no credentials on the server, and the **client pays for the tokens**. That makes sampling the right choice for public servers, which shouldn't run up AI costs for every user.
- Server side: `await ctx.session.create_message(messages=[SamplingMessage(...)], max_tokens=…, system_prompt=…)`. Client side: pass a `sampling_callback` to `ClientSession` that calls Claude and returns a `CreateMessageResult`.

Log and progress notifications

- For long-running tools, the server reports what it's doing through the injected `Context`: `await context.info("…")` for log lines and `await context.report_progress(20, 100)` for progress.
- On the client, `logging_callback` goes on the `ClientSession`, and `progress_callback` goes on each `call_tool()`.
- How you present them is up to you: print them in a CLI, push them over WebSockets, SSE or polling on the web, or drive progress bars on desktop. They're optional and purely for the user's benefit.

Roots: which folders a server may touch

- Roots tell a server which files and folders it may use. Claude can then call `list_roots` and look through those directories, so "convert biking.mp4" works without the user typing a full path.
- They also set a boundary: grant Desktop only, and Documents and Downloads stay off-limits. Requests outside the roots should return an error that Claude can explain to the

user.

Watch out · the SDK doesn't enforce roots

You have to write the check yourself, typically an `is_path_allowed(path)` helper that compares the requested path against the approved roots. Call it in every tool that touches files, before the operation runs.

JSON message types

- The authoritative list is the MCP **specification** repository, separate from the SDKs. It's written in TypeScript purely as a way to describe the data types.
- **Request → Result pairs**: Initialize, CallTool, ListPrompts, ReadResource and so on. **Notifications** are one-way, with no reply: Progress, LoggingMessage, ToolListChanged, ResourceUpdated.
- The protocol is **bidirectional**: servers send requests to clients too (CreateMessage for sampling, ListRoots). That's what makes the choice of transport matter.

Transports

Transport	How it works	Limits
stdio	The client launches the server as a subprocess; the client writes to stdin and the server writes to stdout. Either side can send at any time	Same machine only. It's the "ideal" baseline, and you can test a server by pasting JSON into the terminal
Streamable HTTP	The client POSTs requests. The server returns an `mcp-session-id` header during initialization, which goes on every later request. A GET opens a long-lived **SSE** stream the server uses to push messages	HTTP alone can't carry server-initiated traffic; the SSE workaround makes it possible, and the flags below can disable it

- **Handshake on every connection**: Initialize Request → Initialize Result (the server's capabilities) → Initialized Notification. No other request is allowed before it.
- **A tool call opens a second SSE stream.** The primary GET stream stays open for server-initiated requests and carries **progress notifications**. A per-call stream carries **log messages and the tool result**, then closes.

The two flags that quietly remove features

Setting (default `false`)	Why you'd turn it on	What you lose
`stateless_http=True`	Horizontal scaling behind a load balancer. The GET (SSE) and POST requests may hit different instances, which would otherwise need to coordinate	Session IDs, all server-to-client requests, **sampling**, **progress** and **subscriptions**. The one gain: clients can skip the initialize handshake
`json_response=True`	Simple, non-streaming responses for systems that expect plain JSON	Streaming on POST responses: no intermediate progress or log lines, just the final result

Production trap · works on stdio, breaks over HTTP

A server that showed progress bars, logs and sampling locally loses them when deployed with these flags on. Develop and test with the transport and flags you'll use in production. If your tools depend on sampling or live notifications, pick a different scaling approach or accept reduced functionality.

Hands-on labs

The exam guide recommends building at least one Claude application that uses the API, one or more tools, some prompt and context engineering, and basic security and evals. These seven labs cover that, each in about an hour, in Python or TypeScript. Each gives the goal, the steps, and a "done when" check. There are no full solutions: hitting the traps yourself is how they stick.

1 · A tool-use loop from scratch

D2 · D8 · M2 · P1

Goal: a raw Messages API loop with two tools, `get_current_datetime` and `add_duration`.

1. Write both schemas, each description with a when-to-use and a when-not-to-use sentence.
2. Loop while `stop_reason == "tool_use"`. Append the full assistant content, then *all* `tool_result`s in one user message.
3. Wrap execution in try/except and return `is_error: true` with the message.
4. Log every turn's ids.

Done when: "What day is it 103 days from today?" chains both tools; a forced exception makes Claude react instead of answering wrongly; and your log shows each `tool_use` id matched in the next turn.

2 · A stream handler that survives a drop

D2 · M2

Goal: stream a tool-using response and assemble it safely.

1. Collect blocks by `index`, adding `input_json_delta` fragments to each.
2. Parse tool input only at `content_block_stop`, and append to history only after `message_stop`.
3. Simulate a drop by breaking out of the event loop after a few deltas.

Done when: a simulated drop raises, leaves history unchanged, and a retry from the last complete turn succeeds. Then remove the guard and watch the *next* request fail validation.

3 · A small eval harness

D4 · D6 · M4 · P1

Goal: define "done" for an extraction prompt as a score.

1. Write 10 cases, 3 of them edge cases (two dates, no date, a relative date), each with an expected output.
2. Add a code grader (parse + required fields) and a model grader that returns strengths, weaknesses, reasoning and a score.
3. Hand-label 5 outputs and measure how often the judge agrees with you.
4. Change one thing in the prompt, then re-run.

Done when: you have a recorded baseline, one measured change with a per-case diff, and a judge-agreement number.

4 · A retry wrapper that doesn't make things worse

D2 · D4 · M4

Goal: one place that decides retriable vs. terminal.

1. Retry 429, 529 and 5xx; fail fast on 400, 401, 403 and 404.
2. Honor `retry-after`; otherwise use exponential backoff with jitter and a cap.
3. Raise on `stop_reason: "refusal"`.
4. Decide whether the SDK's own retries stay on, so you never stack two retry loops.

Done when: a faked series of 429s shows growing waits and stops at the cap, a faked 400 raises at once, and a refusal never retries.

5 · Measure prompt caching

D5 · M4 · P1

Goal: see caching work, and see it break.

1. Use `count_tokens` to confirm your system prompt is above the minimum (~1,024 tokens).
2. Put `cache_control: {type: "ephemeral"}` on it and send two requests.
3. Change one character and send again, then wait past the TTL and send again.

Done when: `usage` shows `cache_creation_input_tokens` on the first call, `cache_read_input_tokens` on the second, a miss after the one-character change, and a miss after the TTL.

6 · An MCP server with all three primitives

D8 · P2 · P3 · M3

Goal: a FastMCP server with one tool, a direct and a templated resource, and one prompt.

1. Test it in the Inspector (`mcp dev server.py`).
2. Add `context.report_progress()` to the tool.
3. Connect it to Claude Code with `claude mcp add`.
4. Run it over Streamable HTTP, then turn on `stateless_http`.

Done when: the Inspector lists all three primitives, Claude Code calls the tool, and you've seen progress updates disappear when `stateless_http` is on.

7 · A guardrail hook in Claude Code

D3 · D7 · M3 · M4

Goal: make "never read `.env*`" an enforced rule, not a convention.

1. Add a `PreToolUse` hook with a matcher for Read, Edit and Write. It inspects the path, exits with code 2 and writes the reason to stderr.
2. Add a `PostToolUse` hook that appends each call to an audit log.
3. Add a deny rule in `.claude/settings.json` as a second layer.

Done when: asking Claude to read `.env` is blocked with your reason shown, the attempt appears in the log, and removing the hook still leaves the deny rule blocking it.

Glossary

Accelerator

A build packaged so the next engagement configures it: parameters, documented assumptions, a bundled eval.

Agent SDK

The agent loop behind Claude Code, as a library (`claude-agent-sdk`, `@anthropic-ai/claude-agent-sdk`). Unlike the Anthropic SDK, which is a thin API wrapper, it runs the loop.

Agentic search

The model issues its own queries over several rounds instead of using a pre-built index. No staleness, but more tokens per query.

Compaction

Summarizing history to free context. Only what the summarizer keeps survives.

Context window

The fixed token budget for one request: prompt, history, tools, results and output.

Eval

Cases, expected behaviors and grades that define "done" as a score.

Exponential backoff

Growing, capped waits with jitter between retries; honors `retry-after` when present.

HITL

A human approval step before a consequential or irreversible action.

Hook

A script bound to a Claude Code lifecycle event. It runs every time, unlike a CLAUDE.md instruction.

Integration test

A test of the handoff between two components, where most silent failures hide.

LLM-as-judge

A second model grading open-ended output with a rubric. Trustworthy only once calibrated against human labels.

MCP

Model Context Protocol: servers expose tools, resources and prompts to any MCP client.

Orchestrator-worker

A lead agent fans work out to parallel subagents and synthesizes the results, at about 15× the tokens.

Pinned model ID

A fixed model snapshot, as opposed to an alias that moves.

Prompt injection

Instructions hidden in content the agent reads, then followed as if they were commands.

Retriable vs. terminal

Whether retrying the identical request could plausibly succeed.

Rules file

A `.claude/rules/` file that loads only for matching `paths`.

Skill

A `SKILL.md` procedure loaded on demand when its description matches.

Subagent

An isolated context that returns a summary. It doesn't inherit history or skills, but does inherit permissions.

Trust boundary

A seam where data crosses between components or environments and needs its own control.

Practice questions and answers

M1

1. **A teammate says two runs of the same prompt must return identical text. What is the most accurate reply?**
   A. True, the model is deterministic
   B. Not necessarily: each token is sampled from a probability distribution, so wording can vary even when both answers are correct
   C. Only true when streaming is off
   D. Only true on the largest model
   Answer: B. Sampling makes generation non-deterministic, which is why tests assert on properties and meaning instead of exact text.

2. **Your request's input is already larger than the context window. What happens?**
   A. The oldest turns are silently dropped
   B. The API rejects it with a validation error before generation
   C. You get truncated output with stop_reason model_context_window_exceeded
   D. The model summarizes the history automatically
   Answer: B. An oversized input fails before generation. model_context_window_exceeded is for a request that fit on input but hit the ceiling while generating. Nothing is ever silently dropped.

3. **A short, well-specified classifier already returns the right label zero-shot. What will adding three examples most likely do?**
   A. Raise accuracy substantially
   B. Add token cost to every call for little or no gain
   C. Switch the model being used
   D. Disable sampling
   Answer: B. Add examples when the output shape is wrong, not when zero-shot already works.

4. **You must process 50,000 documents overnight at the lowest cost, with nobody waiting. Which request shape fits?**
   A. A synchronous loop
   B. Streaming
   C. Message Batches with polling
   D. One request with a larger context window
   Answer: C. Batches trade latency for a lower per-token cost and avoid the rate limits a synchronous loop hits. Streaming only helps when someone is watching.

5. **You set temperature to 0.2 on one of the newest Claude models. What happens?**
   A. Outputs become fully deterministic
   B. The request returns a 400; behavior is steered through prompting instead
   C. The value is silently ignored

D. It only changes response length

Answer: B. The newest models don't accept non-default sampling parameters (temperature, top_p, top_k).

6. **Which statement correctly separates model choice from reasoning mode?**

   A. They are the same setting

   B. Extended thinking is a separate model

   C. Model choice picks the family member; reasoning is configured per request, and the two compose

   D. Reasoning mode is fixed per account

   Answer: C. They are independent levers. A capable model with reasoning off is fast and direct; a hard task can pair a capable model with higher effort.

7. **In the TypeScript SDK, how do you make non-blocking calls?**

   A. Import a separate AsyncAnthropic class

   B. The standard client is Promise-based, so you await it

   C. Switch to the Batches API

   D. Call the REST API directly

   Answer: B. Python has AsyncAnthropic. In TypeScript the regular client already returns Promises.

M2

1. **A ticket classifier returns 'Billing', 'billing', or 'This looks like a billing issue.' Which change fixes it?**

   A. Longer descriptions of each category

   B. An output constraint (exact label set, label only), with few-shot examples showing the format

   C. Turning on extended thinking

   D. A sentence asking it to be precise

   Answer: B. Wrong shape means a missing output constraint. Examples in XML tags lock the casing. More description is the classic 'prompt that grew longer' mistake.

2. **Answers start drifting in scope and tone the deeper a conversation runs. What is most likely missing?**

   A. Few-shot examples

   B. A specific system prompt that holds role, scope and format across turns

   C. A JSON schema

   D. A larger max_tokens

   Answer: B. Drift across turns points to an underspecified behavioral contract, which belongs in the system prompt.

3. **Turn 2: tool_use id toolu_01. Turn 3: tool_result with tool_use_id toolu_02. The API rejects the request. What is the fix?**

A. Add an exclusion condition to the tool description
B. Make the tool_result reference toolu_01
C. Add a required array to the input schema
D. Retry the request
Answer: B. Results are matched to calls by id, not position. Routing was correct, so the description isn't the problem.

4. **To save context before the next tool call, you strip thinking blocks out of history. What happens?**
A. Context drops and everything works
B. The signature no longer matches and the API rejects the next request
C. Claude re-thinks from scratch
D. Only redacted blocks need to be kept
Answer: B. Thinking blocks, redacted ones included, go back unchanged. Manage context pressure with context engineering, not by editing blocks.

5. **A stream drops after a tool_use block opened but before content_block_stop. Your handler appends the turn when its read loop ends. What should it do instead?**
A. Parse whatever JSON arrived
B. Append only after message_stop; otherwise discard the partial turn and retry from the last complete turn
C. Append it and fix the schema later
D. Switch to a larger model
Answer: B. A loop ending is not a message completing. A half-built tool_use in history breaks the next request, and the error then points at the wrong place.

6. **Turns 1–4 select tools correctly, each adding a 2,400-token result. From turn 5 the agent picks the wrong tool. Best fix?**
A. Rewrite the apply_coverage_rule description
B. Prune earlier fetch results and compact before the window crowds out instructions
C. Increase max_tokens
D. Switch to parallel tool calls
Answer: B. Correct calls in turns 1–4 rule out the schema. max_tokens controls output length, not how much context the model can read.

7. **Two tools both say 'use this to find information.' Claude keeps picking the wrong one. What is the targeted fix?**
A. Rename one tool
B. Add a when-to-use and a when-not-to-use sentence to each; if they still blur, merge them into one tool with a type parameter
C. Make every parameter required

D. Register more tools so Claude has options

Answer: B. Claude routes mostly on descriptions. Exclusion conditions give it a decision rule.

8. **Your agent will process PHI under a HIPAA BAA. Which wiring path is ruled out?**

   A. A raw Messages API loop on a covered configuration

   B. The Agent SDK on a covered configuration

   C. Claude Managed Agents

   D. Bedrock on a HIPAA-eligible account

   Answer: C. Managed Agents store sessions server-side and are not currently ZDR or HIPAA BAA eligible. The compliance constraint picks the path before convenience does.

9. **A support agent checks in with the same user daily for two weeks, and each session continues the last. Which memory scope fits?**

   A. In-context

   B. External storage, read at session start

   C. Stateless

   D. None; use a bigger window

   Answer: B. In-context state dies at session end. Stateless fits independent jobs. A multi-hour session that won't continue fits in-context.

10. **A reference product diagram is attached to every request your pipeline makes. How should you send it?**

    A. Inline base64 every time

    B. Files API: upload once and reference the file_id

    C. A short-expiry signed URL

    D. Message Batches

    Answer: B. Re-sending base64 inflates every request. Use inline base64 for one-offs, and Batches for volume jobs rather than for encoding images.

11. **With structured outputs (JSON schema) enabled, which responses can still fail to match the schema?**

    A. None; matching is guaranteed

    B. Refusals (stop_reason refusal) and truncations (stop_reason max_tokens)

    C. Only streamed responses

    D. Only responses that use tools

    Answer: B. Constrained decoding rules out malformed JSON, but you still check stop_reason. It also can't be combined with prefill.

12. **A skill works in Claude Code, but in a headless Agent SDK job it never loads. Most likely cause?**

    A. The skill description is too long

    B. settingSources was never set to include filesystem sources

    C. The SDK doesn't support skills

D. Subagents stripped it

Answer: B. Whether CLAUDE.md and skills load in the SDK depends on settingSources. Set it explicitly instead of relying on a default.

13. **Which is a valid reason to choose a workflow over an agent?**

    A. Inputs vary unpredictably

    B. You can write the exact steps in code and want step-level guardrails

    C. The task needs creative tool sequencing

    D. You only know the goal

    Answer: B. Escalate in order: a single call, then a workflow, then an agent. Choose an agent only when the path can't be enumerated.

M3

1. **Trusted payments refactor: auto-approve edits, never run destructive shell commands, never read .env.production. Which settings belong?**

   A. defaultMode bypassPermissions

   B. allow Bash(*) and Edit(*)

   C. allow Bash(npm run:*), deny Bash(rm:*) and Bash(git push:*), plus deny Read(.env.production)

   D. defaultMode default only

   Answer: C. Deny rules gate the destructive commands and the secret file at the settings layer. Bypass removes every prompt, and default prompts on every edit.

2. **In that same refactor, the agent proposes editing a deployment config read by several production services. Where should the human gate go?**

   A. Nowhere; edits are already auto-approved

   B. A human approves the change before the write runs

   C. Review it at the next pull request

   D. Switch to bypassPermissions

   Answer: B. The value is hard to undo and reaches beyond the file, so the gate goes before the write.

3. **Which mode suits a locked-down CI job that should automatically deny anything not on an allow list?**

   A. acceptEdits

   B. plan

   C. dontAsk

   D. auto

   Answer: C. dontAsk runs pre-approved tools plus read-only commands and denies everything else without queuing a prompt.

4. **You want fewer prompts on local work but still want a safety net. Which mode?**

   A. bypassPermissions

B. auto, where a classifier screens each action

C. dontAsk

D. Plan mode

Answer: B. bypass also removes the protected-path guard. auto blocks escalations, prod deploys, mass deletes and force-pushes by default.

5. **A rule should apply only when Claude works on src/db/**/*.sql. Where does it go?**

   A. CLAUDE.md

   B. A .claude/rules/ file with a paths glob in its frontmatter

   C. A subfolder of .claude/rules/ with no frontmatter

   D. The user settings file

   Answer: B. Scoping comes from the paths field, not from folder placement. Without paths, a rules file loads every session.

6. **You must deterministically block any read of .env.production. Which hook?**

   A. PostToolUse that logs the read

   B. PreToolUse that inspects the path and exits with code 2, reason on stderr

   C. SessionStart that prints a warning

   D. UserPromptSubmit that rewrites the prompt

   Answer: B. Only PreToolUse runs before the tool and can block it. PostToolUse runs after the fact.

7. **You delegated a task to the built-in Explore subagent, and it ignored a CLAUDE.md rule. Why?**

   A. Subagents never read files

   B. Explore and Plan skip CLAUDE.md and git status for speed

   C. The rule was too short

   D. Hooks overrode it

   Answer: B. Use the general-purpose subagent, or a custom one that loads the rules it needs.

8. **A plugin skill calls /Users/alexmorgan/projects/deploy-utils/validate.sh. It installs for everyone but fails for teammates. Fix?**

   A. Point it at a shared network drive

   B. Use ~/projects/deploy-utils/validate.sh

   C. Reference the script via $CLAUDE_PROJECT_DIR, or ${CLAUDE_PLUGIN_ROOT} if it's bundled

   D. Delete the step

   Answer: C. Paths must resolve on any machine. Also document required env vars and test the install on a clean machine.

9. **A code-search service hosted on company infrastructure should be available to the whole engineering team. Transport and scope?**

   A. stdio + Local

B. HTTP + Project (.mcp.json)

C. stdio + Project

D. HTTP + Local

Answer: B. Remote and shared means HTTP. Committing .mcp.json shares it through the repo.

10. **IT needs a security-scanning MCP server on every developer's Claude Code install. Transport and scope?**

    A. HTTP + Enterprise managed settings

    B. stdio + User

    C. HTTP + Project

    D. stdio + Local

    Answer: A. Managed settings push it org-wide, and users can't override them.

11. **An API key was committed inline in .mcp.json, then removed in the next commit. What is its status?**

    A. Safe now that the file is fixed

    B. Still in repository history: treat it as compromised, rotate it, and reference ${ENV_VAR} going forward

    C. Safe if the repo is private

    D. Safe after a force-push

    Answer: B. Overwriting a file doesn't clean history. Back it up with a PreToolUse hook that blocks credential-looking writes.

12. **OAuth sign-in works in staging but fails in production with 'redirect URI mismatch'. Fix?**

    A. Switch to API keys

    B. Register the production host's redirect URI, and check whether separate app registrations per environment are required

    C. Retry the sign-in

    D. Rotate the client secret

    Answer: B. Redirect URIs are registered per host. Add the step to the deployment checklist.

13. **CI trace: 401 Unauthorized after reading a plaintext token from /home/jenkins/.config/mcp-credentials.json. Best fix?**

    A. Rotate the key and write the new one to the same file

    B. Rotate the rejected key, move it to a CI-injected env var, and reference the variable in config

    C. Switch the service to OAuth

    D. Retry five more times

    Answer: B. Rotating alone clears the 401 but keeps the storage defect. OAuth doesn't match a service-account identity.

14. **You allow mcp__github__create_issue. What happens to the other GitHub server tools?**

    A. All are allowed

B. They still prompt; a deny on any tool would beat an allow on the server

C. They are hidden from the model

D. They are denied

Answer: B. Per-tool rules narrow a broad server. The API connector's enabled flag is the separate control over whether the model sees a tool at all.

M4

1. **Which grader fits 'Is this summary faithful to the thread?'**

   A. Exact string match

   B. A code check that the output is non-empty

   C. An LLM-as-judge with a rubric, calibrated against human labels

   D. A regex for key phrases

   Answer: C. Open-ended quality needs a judge, and the judge is only defensible once its agreement with human labels has been measured.

2. **The expected output is a JSON array of three capitals, but a correct run lists them in a different order. Which grader scores it correctly?**

   A. Exact match

   B. A code check that parses the JSON and checks membership

   C. LLM-as-judge only

   D. None

   Answer: B. Exact match fails a valid reordering. Code checks are cheap enough to run on every commit.

3. **Your judge returns about 6/10 for nearly everything. What helps most?**

   A. Use a bigger judge model

   B. Ask for strengths, weaknesses and reasoning before the score, then calibrate against human-labeled cases

   C. Average three runs

   D. Switch to exact match

   Answer: B. Without reasoning first, judges drift to a safe middle score. If agreement stays low, tighten the rubric and add examples.

4. **Unit and functional tests pass, but end-to-end fails: retrieve() returns a list of dicts while build_prompt() expects a string. Which level would have caught it?**

   A. Unit

   B. Functional

   C. Integration

   D. None; it's a model issue

   Answer: C. The handoff between components is where silent failures hide. Fix the format contract and add an integration test with real retrieved data.

5. **Which set lists only retriable statuses?**
   A. 400, 401, 429
   B. 429, 529, 500, 503
   C. 403, 404, 529
   D. 400, 500, 504
   Answer: B. 429, 529 and 5xx clear with time. 400, 401, 403 and 404 fail identically on retry.
6. **A response comes back HTTP 200 with stop_reason 'refusal'. What should your code do?**
   A. Retry with backoff
   B. Fail fast: raise it to the caller and log it
   C. Treat the text as valid output
   D. Switch models and retry silently
   Answer: B. A refusal is a content decision, not a transient fault, and a status-code check won't catch it.
7. **A tool throws, and your wrapper returns an empty string as the tool_result. What goes wrong?**
   A. Nothing; Claude ignores empty results
   B. Claude treats the empty result as valid data and builds a confident wrong answer; return is_error: true with a message
   C. The API rejects the request
   D. The loop exits
   Answer: B. A visible failure is far easier to catch than a confident answer built on missing data.
8. **A retry loop catches every Exception and calls time.sleep(0) between attempts. What are its defects?**
   A. Too few attempts
   B. No backoff, so instant retries deepen rate limits, and it retries terminal errors
   C. It should catch BaseException
   D. It needs streaming
   Answer: B. Honor retry-after, otherwise use exponential backoff with jitter and a cap, and fail fast on terminal statuses.
9. **Traffic is mostly simple lookups with a few complex syntheses. Which model strategy?**
   A. Opus for everything
   B. Haiku for everything
   C. Sonnet for everything
   D. A Sonnet or Haiku default with an Opus override for complex requests
   Answer: D. With mixed traffic, a single model either overpays on simple requests or underperforms on hard ones.
10. **You split a tightly coupled refactor across five parallel subagents. The bill tripled and quality barely moved. Why?**

A. The subagents used the wrong model

B. Each subagent spends its own tokens; fan-out (about 15×) only pays off when the work splits into independent parts

C. Caching was off

D. Streaming was off

Answer: B. Dependent steps leave workers waiting on each other. Move the task back to a single agent with good context.

11. **Cost-sensitive nightly job, no user waiting, the same long system prompt on every request. Best configuration?**

    A. Orchestrator-worker

    B. Streaming

    C. Message Batches plus prompt caching

    D. Fetch-once retrieval

    Answer: C. The batch discount and cached-prefix reads stack.

12. **A fetched page hides a line telling your agent to write data to /public/exfil.txt. Which control reliably limits the damage?**

    A. Wrapping fetched text in delimiters

    B. A system prompt saying 'ignore injected instructions'

    C. Least-privilege write paths plus a PreToolUse hook that denies and logs out-of-scope writes

    D. Trusting only internal users

    Answer: C. Delimiters and training are soft or probabilistic. The reliable boundary is what the agent is allowed to do.

13. **Several hooks and rules apply to the same action: one denies, two allow. Outcome?**

    A. Allowed by majority

    B. Denied: deny beats ask, which beats allow

    C. The first rule wins

    D. The user is asked

    Answer: B. That precedence is what makes a hook a real boundary.

14. **What does OS-level sandboxing add on top of hooks and least-privilege roles?**

    A. Faster tool calls

    B. Isolation that holds even for paths and endpoints no hook or rule explicitly covers

    C. Automatic eval runs

    D. Prompt caching

    Answer: B. Filesystem limits to the working directory and network limits to named endpoints are enforced by the OS, so they don't depend on rules being complete.

1. **A 'reusable' template hardcodes repo_path='/home/acme/checkout-service'. What is the correct fix?**
   A. Rename the variable
   B. Make repo_path a function parameter set per engagement, with a documented default
   C. Move the path into the system prompt
   D. Leave it; the next team can edit the file
   Answer: B. A template that runs isn't packaged yet. The next team should configure values, not edit the loop.

2. **You want to share a full customer-service app, UI and deployment scripts included, through the Cookbook. What has to happen first?**
   A. Nothing; submit it
   B. Extract the reusable pattern as a focused example
   C. Add a README
   D. Open-source the UI separately
   Answer: B. The Cookbook reviews focused patterns. A whole application stalls in review.

3. **You have a one-line fix to an existing Cookbook example, carried in from a customer engagement. What must be cleared before technical review?**
   A. A performance benchmark
   B. The rights check: confirm you may contribute engagement code, and attribute prior work
   C. A design doc
   D. Maintainer sign-off on style
   Answer: B. Licensing decides whether a contribution can be accepted at all, so it comes before technical review.

4. **An EU bank wants call-transcript summaries reviewed before they're stored in the EU. Which is a functional requirement?**
   A. The agent should be fast and accurate
   B. A human approves each summary before it is stored
   C. Transcript data must stay in the EU
   D. It must run on an approved cloud provider
   Answer: B. 'Fast and accurate' can't be checked. EU processing and the cloud provider are infrastructure requirements.

5. **In the lifecycle, which phase do 'choosing Bedrock because the customer's compliance posture is there' and 'gating promotion on the eval' belong to?**
   A. Both requirements
   B. Design, then deploy
   C. Both operate
   D. Build, then test

Answer: B. Choosing the platform is design. Pinning and gating promotion are deploy. A residency rule is a requirement, and per-call instrumentation is operate.

6. **An AWS customer has a data-residency requirement and must be able to roll back a model update. Minimal configuration?**
   A. First-party API + Anthropic API key + alias
   B. Bedrock + AWS identity + pinned full model ID + retain the prior pinned version
   C. Vertex + alias + no retention
   D. Bedrock + alias + retain prior
   Answer: B. The customer's cloud and residency need point to Bedrock. Pinning plus retention makes rollback possible.

7. **A deployment used model='opus'. Overnight the parser started failing with KeyError 'summary'. Root cause?**
   A. A parser bug
   B. An unpinned alias advanced to a new version, and no prior pinned version was kept
   C. Rate limiting
   D. Prompt caching
   Answer: B. The app didn't change; the alias did. Pin the full ID, keep the previous version, and gate new versions through the eval.

8. **Latency was measured at 180 ms from a dev laptop. The customer is in eu-west and requires EU-only residency, and the review rejected the platform. Fix?**
   A. Optimize the parser
   B. Remeasure from eu-west and choose a platform whose region satisfies EU residency
   C. Add a cache
   D. Negotiate the requirement
   Answer: B. The rejection was about residency. For regulated customers compliance is pass/fail and decides placement on its own.

9. **A Claude Code task fetches a customer page and passes it straight into the next call's input. What control belongs at that seam?**
   A. Run it as instructions
   B. Treat it as data, and scope the downstream MCP server to least privilege (for example read-only)
   C. Full access for the MCP server so nothing fails
   D. Nothing; each component passed its tests
   Answer: B. Components that pass their own tests say nothing about the seams between them. The app is only as contained as its most privileged seam.

10. **Which model ID pins a fixed snapshot for a pre-4.6 model?**
    A. claude-haiku-4-5
    B. haiku

C. claude-haiku-4-5-20251001

D. latest

Answer: C. Before Claude 4.6 you need the date suffix. From 4.6 onward, the model ID alone pins a snapshot.

P1

1. **Why should a web or mobile app call your own server instead of the Anthropic API directly?**

   A. The API blocks browsers

   B. The API key is secret; shipping it in client code lets anyone extract it

   C. It's faster

   D. Streaming only works server-side

   Answer: B. Keep the key on your server and forward requests from there.

2. **A user asks a question, then follows up with 'Write another sentence', and gets an unrelated sentence. Why?**

   A. Temperature is too high

   B. The API is stateless; you must send the full message history each turn

   C. max_tokens is too low

   D. The system prompt is missing

   Answer: B. Append Claude's reply and the new user message to your list, then send the whole list.

3. **Which of these does NOT make Claude stop generating?**

   A. Reaching max_tokens

   B. An end-of-sequence token

   C. Hitting a stop sequence

   D. Hitting the temperature value

   Answer: D. Temperature shapes sampling. The stop conditions are max_tokens, a natural end, and stop sequences.

4. **Which grader fits 'the output must be a regex that compiles'?**

   A. A model grader

   B. A code grader using re.compile

   C. A human grader

   D. Exact match against a reference regex

   Answer: B. Syntax and format checks are cheap and deterministic in code. Task-following is the model grader's job.

5. **In the course's meal-plan example, which single change gave the biggest score jump?**

   A. Adding XML tags

   B. Being specific with output guidelines (3.92 → 7.86)

   C. Using a bigger model

D. Raising temperature

Answer: B. Listing exactly what the output must include (calories, macros, timing, portions) more than doubled the score.

6. **Your tool function raises ValueError('Location cannot be empty'). Why is that useful?**
   A. It stops the conversation
   B. Claude sees the error message and can retry with corrected arguments
   C. It gets logged by Anthropic
   D. It disables the tool
   Answer: B. Clear errors, returned as tool_result with is_error, let Claude self-correct.
7. **After sending a tool_result, what must the follow-up request still include?**
   A. Only the tool_result
   B. The full history and the tools list again
   C. Just the last assistant message
   D. A new system prompt
   Answer: B. Claude needs the tool definitions to make sense of the tool blocks in the history.
8. **With streaming, tool arguments arrive in delayed bursts. What causes it, and what does fine-grained tool calling change?**
   A. Network jitter; nothing
   B. The API buffers and validates each complete top-level key; fine-grained turns validation off, so chunks arrive sooner and you must handle invalid JSON
   C. Rate limiting; it raises limits
   D. Caching; it disables the cache
   Answer: B. Fine-grained tool calling is faster but leaves you to cope with things like "word_count": undefined.
9. **With the built-in text editor tool, who performs the actual file operations?**
   A. Anthropic's servers
   B. Your application: Claude knows the schema, you implement view, replace, create and so on
   C. Claude Code
   D. The Files API
   Answer: B. Web search and code execution run on Anthropic's side; the text editor tool doesn't.
10. **The code-execution container has no network access. How do you get a CSV into it?**
    A. Paste it as base64 text
    B. Upload it with the Files API and reference it in a container_upload block
    C. Give Claude a URL
    D. Mount a volume
    Answer: B. Generated plots come back as file IDs you download through the Files API.

11. **A semantic search for 'INC-2023-Q4-011' returns related sections but misses the exact one. Best fix?**
    A. Smaller chunks
    B. Add BM25 lexical search and merge the rankings with reciprocal rank fusion
    C. Raise temperature
    D. Switch embedding provider
    Answer: B. BM25 weights rare terms like IDs highly; RRF fuses the two ranked lists.
12. **With RRF and k = 1, a chunk ranked 1st by one index and 2nd by another scores...**
    A. 0.5
    B. 0.833
    C. 1.5
    D. 0.333
    Answer: B. 1/(1+1) + 1/(1+2) = 0.5 + 0.333 = 0.833.
13. **Vector search returns cosine distances of 0.05 and 0.71. Which chunk is more similar to the query?**
    A. 0.71
    B. 0.05
    C. They're equal
    D. You can't tell without similarity
    Answer: B. Cosine distance = 1 − similarity, so lower distance means closer.
14. **Which fields turn on citations for a PDF document block?**
    A. "cite": true
    B. "title" plus "citations": {"enabled": true}
    C. A system prompt asking for sources
    D. A web_search tool
    Answer: B. Responses then include cited_text, the document index and title, and page numbers.
15. **You change the system prompt but keep the same cached tools. What shows up in usage?**
    A. A full cache miss
    B. A cache read for the tools and a cache write for the new system prompt
    C. No change
    D. An error
    Answer: B. The cache works in order: tools, then system, then messages. Only the changed part is rewritten.
16. **In an MCP server, what's the difference between a resource and a tool?**
    A. None
    B. A resource exposes data the app pulls straight into the prompt (like GET); a tool is an action Claude decides to call

C. Resources are only for images

D. Tools can't take parameters

Answer: B. For example, an @document mention reads a resource. A templated URI like docs://documents/{doc_id} passes the parameter to your function.

17. **Which pattern sends each input to exactly one specialized pipeline?**

    A. Parallelization

    B. Chaining

    C. Routing

    D. Evaluator-optimizer

    Answer: C. Parallelization runs every specialist at once and aggregates. Routing picks one.

18. **Which toolset design works best for a general-purpose coding agent?**

    A. Specialized tools like refactor_code and install_deps

    B. Abstract, combinable tools like bash, read, write, edit, glob, grep

    C. A single do_task tool

    D. No tools; rely on the prompt

    Answer: B. Abstract tools let the agent combine them for tasks nobody planned for.

19. **What does the course recommend when choosing between workflows and agents?**

    A. Agents by default

    B. Workflows wherever possible; agents only when truly required

    C. Always both

    D. Whatever is faster to build

    Answer: B. Workflows are more accurate and easier to test. Users want reliability, not a fancy agent.

P2

1. **Which MCP primitive is controlled by the model?**

    A. Resources

    B. Prompts

    C. Tools

    D. Roots

    Answer: C. Claude decides when to call tools. Your app decides when to read resources, and users trigger prompts.

2. **You want users to start a vetted 'summarize this document' workflow from a slash command. Which primitive fits?**

    A. A tool

    B. A resource

    C. A prompt

    D. A sampling request

Answer: C. Prompts are user-controlled templates, surfaced as buttons, menus or slash commands.

3. **Your UI needs the list of documents for an @-mention autocomplete. Which primitive?**
   A. A direct resource such as docs://documents
   B. A tool
   C. A prompt
   D. A templated tool
   Answer: A. Resources are app-controlled data. A templated resource such as docs://documents/{doc_id} then fetches one document.
4. **Which message does an MCP client send to discover what a server offers?**
   A. CallToolRequest
   B. ListToolsRequest
   C. ReadResourceRequest
   D. GetPromptRequest
   Answer: B. ListToolsRequest returns the tools; CallToolRequest runs one with arguments.
5. **How do you test an MCP server's tools without wiring up a full application?**
   A. Unit tests only
   B. Run mcp dev mcp_server.py and use the browser Inspector
   C. Deploy it to Claude Desktop
   D. Call the Anthropic API directly
   Answer: B. The Inspector lets you connect, list and run tools, and check resources and prompts.
6. **With FastMCP, where does a tool's JSON schema come from?**
   A. You write it by hand
   B. It's generated from the decorated function's type hints and Pydantic Field descriptions
   C. Claude infers it at runtime
   D. It's fetched from the Anthropic docs
   Answer: B. The decorator approach removes the manual schema writing from the raw API chapter.

P3

1. **You're publishing a public MCP server whose research tool needs to summarize what it fetched. How should it get the summary generated?**
   A. Give the server its own Anthropic API key
   B. Use sampling: ask the connected client to call Claude, so the client pays for the tokens
   C. Return raw text and hope the user summarizes it
   D. Use a resource
   Answer: B. Sampling keeps credentials and cost off the server, which matters most for public servers.

2. **Where do the logging and progress callbacks go on the client?**
   A. Both on call_tool
   B. logging_callback on the ClientSession; progress_callback on each call_tool
   C. Both in the server's Context
   D. In the system prompt
   Answer: B. The server emits them with context.info() and context.report_progress(); the client decides how to show them.
3. **Does the MCP SDK automatically block a tool from reading files outside the configured roots?**
   A. Yes, always
   B. No; you implement a check such as is_path_allowed() in every file-touching tool
   C. Only on stdio
   D. Only when Claude asks
   Answer: B. Roots tell the server which folders it may use, but enforcement is your code's job.
4. **What must happen before a client can call a tool on a stateful connection?**
   A. Nothing
   B. The Initialize Request → Initialize Result → Initialized Notification handshake
   C. A ListToolsRequest
   D. A sampling request
   Answer: B. Only stateless HTTP lets clients skip initialization.
5. **Which message type expects no response?**
   A. CallToolRequest
   B. InitializeRequest
   C. ProgressNotification
   D. ReadResourceRequest
   Answer: C. Notifications are one-way. Requests always pair with a Result.
6. **Over Streamable HTTP, how can the server send messages to the client at all?**
   A. It can't
   B. The client opens a long-lived GET Server-Sent Events stream the server pushes on, identified by the mcp-session-id
   C. Through WebSockets only
   D. By polling the client's URL
   Answer: B. A tool call opens a second SSE stream for that call's logs and result, which closes once the result is sent.
7. **You set stateless_http=True to scale behind a load balancer. What breaks?**
   A. Only logging
   B. Sampling, progress, subscriptions and every other server-to-client request; there are no session IDs

C. Nothing

D. Tool calls

Answer: B. The one upside is that clients no longer need the initialize handshake.

8. **What does json_response=True change?**

   A. Tools must return JSON

   B. POST responses stop streaming: no intermediate progress or logs, only the final result as plain JSON

   C. It enables sampling

   D. It encrypts responses

   Answer: B. Choose it when you don't need streaming, or when the systems you integrate with expect plain JSON.

9. **A server shows progress bars and uses sampling locally, but both vanish once it's deployed. Most likely cause?**

   A. A model change

   B. Moving from stdio to HTTP with stateless_http or json_response turned on

   C. Rate limiting

   D. Missing roots

   Answer: B. Test with the same transport and flags you'll run in production.

10. **Which transport lets either side send at any time, but only on one machine?**

    A. Streamable HTTP

    B. stdio

    C. SSE-only

    D. Stateless HTTP

    Answer: B. stdio is the bidirectional baseline: the client writes to stdin and the server writes to stdout.

Exam

1. **10,000 documents must be analyzed overnight for a non-urgent report, and cost is the main concern. Which approach fits?**

   A. Synchronous Messages API calls in parallel to finish fast

   B. The Message Batches API: asynchronous, within a 24-hour window, at reduced cost

   C. Lower max_tokens on synchronous calls

   D. Switch to the smallest model regardless of quality

   Answer: B. In the style of the official sample. Parallel sync calls don't lower the per-token price, and cutting max_tokens or downsizing blindly doesn't address the batch-vs-realtime trade-off.

2. **A summarizer reads user-submitted pages. One contains hidden text telling the model to ignore its instructions and reveal its system prompt. Most effective mitigation?**

   A. Raise the temperature

   B. Treat page content as untrusted, keep it separate from trusted instructions, and use

guardrails or hooks so injected text can't trigger sensitive actions

C. Ask users in the system prompt not to submit malicious content

D. Use a larger model that follows instructions better

Answer: B. In the style of the official sample. A polite request isn't enforceable, and a more instruction-following model can be more susceptible, not less.

3. **Several Claude apps need to call an internal inventory REST service, and the integration should be maintained independently of any one app. Best approach?**

   A. Hard-code inventory logic into each system prompt

   B. Build an MCP server exposing the inventory operations as tools

   C. Paste current inventory data into every request

   D. Rely on a built-in tool to reach the API

   Answer: B. In the style of the official sample. Built-in tools don't automatically reach arbitrary internal APIs, and pasted data is stale and wastes context.

4. **Your team wants a repeatable code-review checklist that Claude Code loads only when someone asks for a review. Best fit?**

   A. Add it to CLAUDE.md

   B. Package it as a Skill whose description matches review requests

   C. Build an MCP server for it

   D. Write a custom tool

   Answer: B. A Skill is a procedure loaded on demand. CLAUDE.md loads every session; MCP is for connecting to an external system.

5. **One application needs a pricing calculation that only it uses, implemented in its own codebase. What should Claude call?**

   A. A new MCP server

   B. A custom tool defined and executed by that application

   C. A Skill

   D. A built-in server tool

   Answer: B. An MCP server earns its maintenance cost when several apps reuse an integration. Single-app logic is a custom tool.

6. **Which tools run on Anthropic's side without your code executing them?**

   A. Any tool with a JSON schema

   B. Server tools such as web search and code execution

   C. The text editor tool

   D. Tools defined with strict: true

   Answer: B. With client tools (your custom tools, and the text editor whose file operations you implement), your code runs the tool and returns a tool_result.

7. **A CI job must run Claude Code non-interactively and parse the result. What do you use?**

   A. An interactive session left open

B. Headless (print) mode, e.g. claude -p with JSON output

C. bypassPermissions in a terminal

D. The Claude desktop app

Answer: B. Headless mode runs a single prompt and returns output for scripts. Pair it with a locked-down permission mode such as dontAsk in CI.

8. **An AI code review of your PR claims a 'race condition in the billing service' that the diff doesn't touch. What do you do?**

   A. Apply its fix immediately

   B. Treat it as a hypothesis: reproduce or test it before acting, and trust only findings the diff itself proves

   C. Ignore all AI review findings

   D. Merge, then investigate

   Answer: B. Triage AI findings. Claims about runtime behavior or other systems need evidence, and a human gate belongs where a finding becomes a hard-to-undo action.

9. **A prompt edit silently changed the output shape in production. How do you make prompt changes reviewable and reversible?**

   A. Edit prompts directly in the production console

   B. Version prompts with the code, pin the model ID, and gate promotion on the eval against a pinned baseline

   C. Use a model alias so you always get the latest

   D. Raise temperature to compensate

   Answer: B. Configuration management covers prompt versioning and model pinning, so every change can be reviewed and rolled back.

10. **You upgrade to a newer Claude model and requests start returning 400. The code sets temperature=0.3 and a thinking budget_tokens. Most likely fix?**

    A. Retry with backoff

    B. Remove the non-default sampling parameter and move to adaptive thinking with an effort setting, then re-run your evals

    C. Lower max_tokens

    D. Switch the SDK language

    Answer: B. A 400 is terminal. Breaking parameter changes across model releases are on the blueprint.

11. **Your team prefers LangGraph or Strands over the Claude Agent SDK. What stays true underneath?**

    A. Nothing; frameworks change how Claude works

    B. The tool-use loop rules: tool_result answers each tool_use by id, thinking blocks go back unchanged, and the context budget still applies

    C. Tools no longer need schemas

D. Evals become unnecessary

Answer: B. Agentic frameworks are abstractions over the same Messages API mechanics.

12. **Claude returns well-formed JSON with a confident but unverified total that downstream code will act on. Best practice?**

    A. Trust it because it parsed

    B. Validate defensively: check the schema and values or ranges, cross-check against the source, and handle refusal and max_tokens

    C. Ask Claude if it's sure

    D. Raise effort to max

    Answer: B. Output handling on the blueprint includes defensive parsing and skepticism toward confident output.

13. **Support transcripts containing customer PII are sent to Claude and logged. Which practice fits secure-by-design?**

    A. Rely on the model to ignore PII

    B. Minimize or redact PII before prompts and logs, restrict log access, and choose a ZDR or BAA setup where the data requires it

    C. Store logs in a shared bucket for debugging

    D. Encrypt only the prompts, not the logs

    Answer: B. Data leakage prevention and PII handling sit under AI Application Security.

14. **A Python web request needs 20 independent Claude calls while the user waits. Which pattern?**

    A. Message Batches API

    B. AsyncAnthropic with concurrent awaits (e.g. asyncio.gather), bounded to stay under rate limits

    C. A synchronous for-loop

    D. One giant prompt

    Answer: B. A user is waiting, so Batches is out. Async gives concurrency without blocking; a cap keeps you under rate limits.

15. **After a long debugging session in Claude Code, you switch to an unrelated feature. Best session hygiene?**

    A. Keep going in the same session

    B. /clear to start fresh, keeping durable rules in CLAUDE.md

    C. /compact and continue

    D. Restart the computer

    Answer: B. Clear when the next task is unrelated; compact when you're continuing the same task near the limit.

Condensed from the Claude Certified Developer – Foundations Prep Course on Anthropic's partner Skilljar (five modules), plus the optional Building with the Claude API, Introduction to Model Context Protocol and Model Context Protocol: Advanced Topics courses, all read 26 October 2026. Path questions are rewritten from the course's quizzes and checkpoints; questions for the optional courses are written from the lesson summaries, because their quizzes load interactively. Product specifics change often; confirm model IDs, headers, limits and pricing on platform.claude.com and code.claude.com.